

JNPF V5.2 全栈底层架构迭代开发计划（完整版）

版本：v5.0（专家审阅裁定版·全平台重构）

编制：首席架构师（整合 D 爷 7/8/9 确定稿 + v4.0 + 2026-06-12 顶级专家审阅裁定）

日期：2026-06-12

状态：待创始人 / 专家组批准

关联 ADR：ADR-016（模块化单体）、ADR-017（新旧代码生成器共存）、ADR-018（UniApp UI 库选型）

草稿来源：7、D爷初稿·8、D爷确定稿第一部分（Baobab-Studio）·9、D爷确定稿第二部分（Baobab-Foundry）

文档结构：第一篇 F-0~F-10 工程基座与编译器·第二篇 自博弈 AI 低代码产品确定版·附录 废止说明与审核清单

v5.0 变更摘要：Foundry 执行引擎 RL→LLM Agentic Loop（~16 周）；Studio 知识图谱 仅 SQL Server；uni-app X 暂缓非删除；新增 FlowIR / AI Evals / 组件覆盖 ≥90% 门禁；详见附录 D

v5.0 修订原则（强制，继承 v4.0）

- 战略不变：四角色闭环、因果图蒸馏、IR 契约、Foundry 物理分离、五阶段流水线——全部保留。
- 引擎换道：Foundry 废弃 MCTS/PP0/A3C/GPU 训练栈，采用 LLM Agentic Loop + RAG + 因果回放池（专利叙事不变）。
- 知识图谱：Studio 侧 **BASE_KNOWLEDGE_**（SQL Server）为唯一运行时**；Foundry MVP2 再评估 Neo4j（创始人裁定：部分采纳，非删除）。
- 移动编译：阶段三 **单轨标准 uni-app**（小程序 + App）；uni-app X 双轨 **暂缓、保留 IR 扩展位**（创始人裁定：非删除）。
- 回写收敛：官方通道仅 **ir-to-schema.ts**；取消任意手改 .vue → IR 反解析；受保护区块 `@jnpf-block` 为后续可选。
- 新增 P0：FlowIR（工作流）、AI Evals golden set、组件 registry 覆盖率 ≥90%（阶段五启动门禁）。
- 基础设施：沙箱共享 SQL Server（per-tenant DB）；Patch 通道 HTTPS + 签名 zip（非 mTLS/WORM）；IR 契约导出 JSON Schema（非 T4）。

计划总纲

本计划整合五条工作线 + 两条前置 Sprint：

- 工作线 A：后端清零（App.GetService / CreateScope / JwtHandler 路由权限 / Outbox 事务）
工作线 B：前端基础层（IR + 表达式引擎 + 组件注册表 + 多目标编译器 + 端到端验证）
工作线 C：数字大屏升级（F-6a 基础 + F-6b 完整 3D 数字孪生 VIP，4 周全量）
工作线 D：UniApp **单轨**（标准 uni-app 小程序 + App；uni-app X **暂缓**，保留 IR 扩展）
工作线 E：Baobab-Studio AI 原生层（五阶段流水线 + 多角色 web UI + 多租户沙箱）
工作线 F：Baobab-Foundry 自博弈（LLM Agentic Loop + 四 Agent + 蒸馏师，~16 周）
工作线 G：**P0 新增** FlowIR 工作流 IR + AI Evals 基准 + 组件覆盖 ≥90%

前置 Sprint 0-A：闭合 Sprint（5 天，工程底座 + 安全 PoC + Schema 门禁）

前置 Sprint 0-B：AI 基础设施地桩（5 天，10 项地桩 + 后端 LlmGatewayService）

文档分工：

第一篇（上文）= F-0~F-10 可执行施工包（编译器、后端清零、代码路径）

第二篇（文末）= 自博弈 AI 低代码确定版（D 爷 7/8/9 升格，产品/API/双系统拓扑）

执行策略：

Sprint 0-A 通过 10 项门禁 → Sprint 0-B 通过 8 项补充门禁 → 阶段一~四并行 F-6a

PoC 门禁：**Three.js**（阶段二前）；uni-app X PoC **暂缓**（单轨先行，生态成熟再议）

阶段五启动前：**组件 registry 覆盖率 ≥90%** + **FlowIR v1** + **Evals 50 条 golden set**

Foundry 仅在独立仓库部署；Studio 经 KnowledgePatch 接收知识（SQL Server 表）

全局时间线

阶段	名称	工期
✅ 阶段零	F-0~F-4 + ADR-016 + src/core 83 tests	已完成
Sprint 0-A	0-A 闭合 Sprint（P0 工程/安全门禁）	5 工作日
Sprint 0-B	0-B AI 基础设施地桩（10 项）	5 工作日（与 F-6a 可并行）
PoC 门禁	Three.js 性能基线（门禁） uni-app X PoC **暂缓**（单轨先行）	1 周（阶段二启动前） 未来生态成熟再评估
阶段一	F-5 收官验证 + F-6a 大屏基础 + 后端 Sprint	4 周（不变）
阶段二	F-6b 完整 3D 数字孪生 VIP + 后端 Sprint	4 周（不压缩 MVP）
阶段三	F-7 **单轨** UniApp 编译器 + FlowIR v1 + 后端清零收尾（App.GetService 业务层）	**4 周**（uni-app X 暂缓）
阶段四	F-8 统一网关 + 下载 + ir-to-schema 回写	3 周
阶段五	五阶段 AI 流水线 + Evals + 多角色 UI	10 周（组件覆盖 ≥90% 门禁）
阶段六	多租户沙箱 + 创始人管理 + Foundry 对接	8 周（Studio 侧）

主体项目（Studio + 代码生成）：Sprint 0-A/B + PoC + 阶段一~六 ≈ **49 周**
Baobab-Foundry（LLM Agentic Loop，独立部署）：**~16 周** 并行（第二篇 §4）

说明：v2.0「46 周压缩为 28 周」及 8 稿「28 周 Studio」**均已废止**；
v5.0 吸收专家审阅：Foundry ~16 周、阶段三 -1 周（单轨 uniApp）、引擎换道。

阶段零回顾（已完成）

- ✅ F-0 安全止血
7 个文件修改：eval/Function 归零 + 密钥外移
标签：无（安全修复，立即合并主分支）
- ✅ F-1 IR 类型系统 + Schema 清洗器
6 个源文件：types.ts, component-mapping.ts, expression-classifier.ts, validator.ts, schema-cleaner.ts, test
AI 探针：aiHints + intentHints + ai-quality
标签：v5.2-ai-ready-m1
- ✅ F-2 安全表达式引擎
11 个文件：tokenizer + parser + security + compiler + functions + context + engine + compat + 3 个测试文件
35 个测试全部通过
标签：v5.2-ai-ready-m2
- ✅ F-3 组件注册表

5 个文件: types + registry + builtin + index + test
35 个内置组件 (10 个分类)
标签: v5.2-ai-ready-m3

✅ F-4 Vue3 单表 CRUD 编译器
8 个文件: types + type-gen + api-gen + list-gen + form-gen
+ hook-gen + compiler + test
11 个编译器测试 + src/core 合计 **83 项 vitest 通过** (canonical, CI 为准)
标签: v5.2-ai-ready-m4

✅ F-5 端到端验证 (full-pipeline.test.ts, 4 项 e2e 测试)
标签: v5.2-ai-ready-m5

✅ 知识图谱文档 (4 份) + ADR-016 模块化单体架构决策

🕒 后端清零 Sprint 1 进行中 (App.GetService 消化, 目标 37→0)

⚠️ v3.0 强制前置: Sprint 0-A / 0-B 门禁通过前, 不得启动阶段一对外里程碑

Sprint 0-A: 闭合 Sprint (5 工作日, P0 工程与安全门禁)

目标: 将多轮架构审查识别的 P0 缺陷从「纸面方案」变为「CI 跑绿」。
不阻塞: F-6a 大屏编译器可与 Day 3 起并行, 但**阶段一里程碑**依赖本 Sprint 全绿。

0-A.1 工程化底座 (Day 1)

任务	路径 / 命令	验收标准
补充 scripts	jnpf-web-vue3/package.json	存在 lint (Day 1 修正: 当前 CI 误调 pnpm lint, 实际脚本为 lint:eslint)、type-check、test:unit、diff:codegen
vitest 配置	jnpf-web-vue3/vitest.config.ts	pnpm test:unit 覆盖 src/core/**
CI 合并修正	.github/workflows/ci.yml	合并 现有 backend/datascreen job; web-vue3 job 去掉 continue-on-error; 脚本名改为 lint:eslint; 新增 test:unit + type:check
锁文件	三前端项目	单一真源策略: 各项目保留独立 pnpm-lock.yaml + working-directory 安装 (非 monorepo filter)

```
cd jnpf-web-vue3 && pnpm lint && pnpm type-check && pnpm test:unit
# 预期: 0 error; tests 83 passed
```

0-A.2 Schema 回归 + 覆盖缺口 (Day 2)

任务	路径	验收标准
P0 fixtures 入库	src/core/ir/__tests__/fixtures/schema-*.json	至少 5 份生产级 Schema (含子表、customBtns、接口回调)
回归测试	src/core/e2e/schema-regression.test.ts	P0 失败必须 red (禁止 console.warn 静默通过)
缺口报告	docs/coverage-gap-report.md	自动生成或手工首版; 阶段五启动门禁: registry 覆盖率 ≥90%
多租户启用 PoC	App.json MultiTenancy + ITenantFilter 越权测试	当前 MultiTenancy=false; Sprint 0-A 登记启用路线图, 阶段六前必须 true

0-A.3 后端安全 PoC (Day 3)

任务	路径	验收标准
Outbox PoC 2	backend/tests/JNPF.Tests.Gate/Infrastructure/OutboxSqlServerPoC.cs	4 用例; 测试 ISqlSugarClient.CopyNew() 与主事务隔离 (SQL Server, 非实体 CopyNew)
JwtHandler 最小路由权限	backend/application/JNPF.API.Entry/Handlers/JwtHandler.cs	路由级匹配 + 403; 保留权限组校验为第二道门
集成测试	backend/tests/JNPF.Tests.Gate/Auth/JwtHandlerIntegrationTests.cs	3 passed
安全债务登记	docs/security-debt-registry.yaml	SD-001 ~ SD-007 (含 QueryFilter Clear/Add 审计, 非 Filter(null) 臆造)

dotnet test backend/tests/JNPF.Tests.Gate --filter "OutboxSqlServerPoC|JwtHandler"

0-A.4 ADR + 映射 + diff (Day 4)

交付物	说明
docs/adr/ADR-017-new-old-codegen-coexistence.md	在线 .vm / 下载 TS 编译器; 附录 diff 脚本
docs/adr/ADR-018-unapp-ui-library-selection.md	新编译器 wot-design-uni; legacyApp 保留 uni_modules
jnpf-web-vue3/src/core/ir/component-mapping.ts	pc / app(wd-) / legacyApp(uni-) 三层
jnpf-web-vue3/scripts/diff-codegen.ts	ESM; 输出 docs/adr-017-diff-report.md
jnpf-web-vue3/src/core/compiler/uniapp/templates/request.ts	Alova 对齐 ResultEnum (200/600/601/602 + HTTP 401)

0-A.5 登记 + Husky + 终验 (Day 5)

任务	路径
进度登记	docs/progress-registry.yaml (canonical tests: 83)
app 基线	docs/app-vue3-baseline.md
Git hooks	jnpf-web-vue3/.husky/ + commitlint.config.js
标签	v5.2-gate-p0-complete

Sprint 0-A 门禁 (10 项, 全部 才进入 0-B)

1. CI 无 continue-on-error (lint/test)
 2. vitest 进 CI
 3. Husky + commitlint 激活
 4. P0 Schema 回归全绿
 5. src/core tests ≥83 passed
 6. OutboxSqlServerPoC 4 passed (SQL Server)
 7. JwtHandlerIntegration 3 passed
 8. Alova request.ts 含 TOKEN_TIMEOUT=600 语义
 9. pnpm diff:codegen 可执行
 10. security-debt-registry.yaml 含 SD-001
-

Sprint 0-B: AI 基础设施地桩 (5 工作日)

目标: 为 Baobab-Studio Phase 1 五阶段流水线预埋数据面与网关。
知识图谱 (创始人裁定): Studio 侧 **SQL Server BASE_KNOWLEDGE_* 为唯一运行时**; Foundry 侧 Neo4j MVP2 再评估 (非删除, 见第二篇 §4.2)。
因果回放池: SQL Server JSON 列 (**废止** PostgreSQL + pgvector)。

10 项地桩清单

# 地桩	核心表 / 路径	Phase
1 AI 调用日志	BASE_AI_CALL_LOG + AiCallLogService (DynamicApi)	0-B Day 6
2 IR aiHints	types.ts / dashboard-types.ts (✅ 已完成, 登记)	—
3 知识图谱存储	BASE_KNOWLEDGE_NODE + BASE_KNOWLEDGE_EDGE + IKnowledgeGraphStore (Sql 实现, 唯一真源)	0-B Day 8
4 五阶段流水线状态	BASE_AI_PIPELINE + BASE_AI_PIPELINE_MESSAGE	0-B Day 6
5 创始人认证	BASE_FOUNDER_AUTH_LOG + FounderGuardMiddleware (Phase 0 404, Phase 3 403)	0-B Day 8
6 Prompt 模板	BASE_AI_PROMPT_TEMPLATE	0-B Day 7
7 前端 AI 骨架	src/ai/gateway/, src/views/studio/	0-B Day 9
8 IR 逆向 (逃生舱)	ir-to-schema.ts + round-trip 测试; 同步导出 ir.schema.json (JSON Schema 契约, TS/C# 双端生成)	0-B Day 7-8
9 后端 LlmGatewayService	modularity/.../LlmGatewayService + 写 BASE_AI_CALL_LOG	0-B Day 6
10 KnowledgePatch 签名	PackageHash + Signature 字段 + 验证接口	0-B Day 8

实体规范 (升标, 强制): 所有新表使用 F_ 列名、string 用户 ID、租户基类 / ITenantFilter; 禁止裸 PascalCase 列映射。

Sprint 0-B 补充门禁 (8 项)

- 11. BASE_AI_CALL_LOG 表存在
- 12. BASE_AI_PIPELINE 表存在
- 13. IKnowledgeGraphStore 接口 + Sql 实现
- 14. FounderGuard 已注册 (/api/founder Phase 0 返回 404)
- 15. src/ai/gateway/types.ts 存在
- 16. LlmGatewayService healthCheck + 写日志
- 17. ir-to-schema 最小 round-trip 1 Schema
- 18. KnowledgeIncrementPackage 含 Signature 字段定义

标签: v5.2-ai-infrastructure-m0 (依赖 v5.2-gate-p0-complete)

PoC 门禁 (阶段二启动前, 1 周)

PoC 内容	通过	未通过 (须创始人书面决策)
PoC-A uni-app X (.uvue HBuilderX 编译)	暂缓 (v5.0 单轨先行)	保留 IR 双轨扩展位; 生态成熟后再启 PoC-A

PoC 内容	通过	未通过 (须创始人书面决策)
PoC- Three.js 10 万面 + 20 POI + 5 飞线, 16G 本 B 机 ≥30fps 10min	排入阶段二全量 F-6b	LOD/面数限制或 2.5D 降级; 不删除 VIP 模块规划

IR 通用性契约 (D 爷裁定, v3.0 强制)

唯一真源: jnpf-web-vue3/src/core/ir/types.ts
正向: JNPF Schema → schema-cleaner.ts → FormPageIR (已实现)
逆向: FormPageIR → ir-to-schema.ts → VisualDev 可编辑 Schema (Sprint 0-B 地桩 8)
验收: 10+ 真实 Schema round-trip diff; AI 产出不可清洗 = AI 错误, 非编译器错误
逃生舱: 五阶段每阶段结束可「转入 VisualDev 手工继续」(需 IR 逆向)
契约导出: `types.ts` → ****ir.schema.json**** (JSON Schema); C# 侧 NJsonSchema 生成 (**废止** T4 同步)
结构化输出: LlmGateway 调用时将 ir.schema.json 作为 response schema (json mode)
生成→验证→自修复: LLM 产出 IR → validator.ts + vue-tsc 程序化打回 → 自动重试 N 轮

专家审阅裁定 · P0 新增施工包 (v5.0)

来源: 2026-06-12 顶级专家审阅 + 总架构师裁定 (详见文档末尾「总架构师意见」)。本节为阶段三~五硬门
禁。

F-7.9 FlowIR 工作流 IR (阶段三 W3-W4, 与单轨 UniApp 并行)

项 说明
动机 JNPF 核心资产为 18 张 FLOW_* 表 + FlowTaskManager; 无 FlowIR 则 AI 生成系统无审批流
交付 src/core/ir/flow-types.ts; FlowTemplateUtil 映射层; FlowIR → 工作流 JSON 编译器 v1
验收 1 条真实审批流 round-trip; 与 FormPageIR 联合编译通过 e2e
门禁 阶段五启动前 FlowIR v1 必须存在

F-5.2 AI Evals 基准 (Sprint 0-B 起建, 阶段五前 ≥50 条)

项 说明
动机 2026 年 AI 产品第一工程资产; 换模型/改 prompt 防回退
交付 src/core/evals/golden/ (50~100 真实需求 → 预期 IR) ; eval-runner.ts 评分脚本
验收 CI 可选 job; 阶段五前 baseline score 登记; 每次 prompt 变更 diff 报告
门禁 阶段五启动前 golden set ≥50 且 runner 可执行

组件覆盖率门禁 (阶段五启动条件)

registry 已注册 jnpfkey / 在线 componentMap 总数 ≥ 90%
数据源: docs/coverage-gap-report.md (Sprint 0-A Day 2 首版)
当前缺口: ~35 vs 60+ (0-2 升级为硬门禁, 非 backlog)

阶段一：收官验证 + 大屏基础升级（4 周）

目标

完成前端基础层的端到端验证（F-5），
同时启动数字大屏的基础技术升级（F-6a），
后端清零 Sprint 1-3 并行推进。

Week 1：端到端验证

F-5.1 验证链路

任务：验证从"平台 JSON Schema"到"可独立运行的 Vue 3 项目"的完整链路

输入：Phase 0 抓取的真实 Schema JSON

管线：Schema → cleanSchema() → FormPageIR → Vue3Compiler.compile() → GeneratedProject

验证：GeneratedProject 写入临时目录 → pnpm install → pnpm build → 构建成功

文件：src/core/e2e/full-pipeline.test.ts

```
// src/core/e2e/full-pipeline.test.ts
import { describe, it, expect } from 'vitest';
import { cleanSchema } from '../ir/schema-cleaner';
import { Vue3Compiler } from '../compiler/vue3/compiler';
import { validateIR, hasErrors } from '../ir/validator';
import minimalSchema from '../ir/__tests__/fixtures/minimal-form-schema.json';

describe('End-to-End Pipeline', () => {
  it('Schema → IR → Compiler → GeneratedProject 完整链路', () => {
    // Step 1: 清洗
    const ir = cleanSchema(minimalSchema);
    expect(ir.type).toBe('form');
    expect(ir.fields.length).toBeGreaterThan(0);

    // Step 2: 验证
    const issues = validateIR(ir);
    expect(hasErrors(issues)).toBe(false);

    // Step 3: 编译
    const compiler = new Vue3Compiler({
      entity: 'test-entity',
      entityLabel: '测试实体',
    });
    const result = compiler.compile(ir);

    // Step 4: 检查生成产物
    expect(result.project.size).toBeGreaterThan(0);
    expect(result.project.has('src/types/test-entity.ts')).toBe(true);
    expect(result.project.has('src/api/test-entity.ts')).toBe(true);
    expect(result.project.has('src/views/test-entity/index.vue')).toBe(true);
    expect(result.project.has('src/views/test-entity/form.vue')).toBe(true);

    // Step 5: 检查生成代码质量
    for (const [path, content] of result.project) {
      // 零 eval/Function
      expect(content).not.toMatch(/\beval\b/);
      expect(content).not.toMatch(/new Function/);
      // 生成标记存在
    }
  });
});
```

```

    expect(content).toContain('@jnpf-generated');
    // insert-point 存在
    if (path.endsWith('.vue')) {
      expect(content).toContain('@jnpf-gen:insert-point');
    }
  }
}

// Step 6: 检查 TypeScript 类型（对生成的 types 文件做语法检查）
const typesContent = result.project.get('src/types/test-entity.ts')!;
expect(typesContent).toContain('export interface');
});
});

```

F-5.2 生成代码演示项目

任务：用"学生管理" Schema 生成完整的可运行 Vue 3 项目

1. 用 cleanSchema() 清洗学生管理 Schema
2. 用 vue3Compiler 编译为 GeneratedProject
3. 用脚本将 GeneratedProject 写入 examples/generated-student/
4. 手动执行 `pnpm install && pnpm dev`
5. 浏览器打开，验证页面可渲染

文件：

```

examples/generated-student/
├─ package.json
├─ vite.config.ts
├─ tsconfig.json
├─ src/
│   ├─ types/student.ts
│   ├─ api/student.ts
│   ├─ views/student/index.vue
│   ├─ views/student/columns.ts
│   ├─ views/student/search.ts
│   ├─ views/student/form.vue
│   └─ composables/useStudent.ts
└─ App.vue
└─ README.md

```

F-5.3 ESLint 规则注入

任务：在 ESLint 配置中新增 eval/Function 零新增规则

文件：.eslintrc.cjs（修改）

内容：

```

rules: {
  'no-eval': 'error',
  'no-new-func': 'error',
  'no-implied-eval': 'error',
}

```

验证：故意写一行 eval()，ESLint 报错

F-5 交付物

- src/core/e2e/full-pipeline.test.ts - 端到端测试
- examples/generated-student/ - 演示项目（可独立运行）
- .eslintrc.cjs - eval/Function 零新增规则
- 标签：v5.2-ai-ready-m5

Week 2-3: 数字大屏基础升级

F-6a.1 大屏 IR 定义

文件: src/core/ir/dashboard-types.ts

内容: DashboardIR, DashboardWidget, DashboardDataSource

```
/**
 * 数字大屏 IR 类型定义
 *
 * 与 FormPageIR 同级，都是 PageIR 的联合类型
 */

export interface DashboardIR {
  type: 'dashboard';
  id: string;
  name: string;

  /** 设计尺寸（如 { width: 1920, height: 1080 }） */
  size: { width: number; height: number };

  /** 背景配置 */
  background: {
    type: 'color' | 'image' | 'gradient';
    value: string;
  };

  /** 主题标识 */
  theme: string;

  /** 组件列表（绝对定位） */
  widgets: DashboardWidget[];

  /** 数据源 */
  dataSources: DashboardDataSource[];

  /** AI 探针 */
  aiHints?: {
    domain?: string;
    scenario?: string;
    designRationale?: string;
  };
}

export interface DashboardWidget {
  id: string;

  /** 组件类型 */
  type: string; // 'chart:bar' | 'chart:line' | 'chart:pie' | 'border:box1' |
                // 'text' | 'image' | 'scroll-board' | '3d:scene' | '3d:poi' |
                // '3d:flyline' | '3d:fence' | '3d:heatmap'

  /** 绝对定位 */
  position: {
    x: number;
    y: number;
    w: number;
    h: number;
    zIndex?: number;
  };
}
```

```

};

/** 组件属性（图表配置、装饰样式等） */
props: Record<string, unknown>;

/** 绑定的数据源 ID */
dataSourceId?: string;

/** 刷新闻隔（ms），0 或 undefined 表示不刷新 */
refreshInterval?: number;

/** 是否可见（支持表达式控制） */
visible?: string;

/** AI 探针 */
aiHints?: {
  purpose?: string;
  dataExpectation?: string;
};
}

export interface DashboardDataSource {
  id: string;
  name: string;
  type: 'api' | 'websocket' | 'static' | 'mock';
  url?: string;
  method?: 'GET' | 'POST';
  params?: Record<string, unknown>;
  /** 轮询间隔（ms），仅 type=api 时有效 */
  pollInterval?: number;
  /** 数据转换表达式 */
  transform?: string;
  /** 静态数据（type=static 时使用） */
  staticData?: unknown;
}

```

F-6a.2 大屏组件扩展注册

文件：src/core/component-registry/builtin-dashboard.ts

内容：注册所有大屏基础组件（图表、装饰、布局、文字）

```

import type { ComponentEntry } from './types';

export const DASHBOARD_COMPONENTS: ComponentEntry[] = [
  // =====
  // 图表组件
  // =====
  {
    type: 'chart:bar',
    name: '柱状图',
    category: 'chart',
    pc: 'echarts-bar',
    app: 'echarts-bar',
    version: '1.0.0',
  },
  {
    type: 'chart:line',
    name: '折线图',
    category: 'chart',
    pc: 'echarts-line',
  },

```

```
    app: 'echarts-line',
    version: '1.0.0',
  },
  {
    type: 'chart:pie',
    name: '饼图',
    category: 'chart',
    pc: 'echarts-pie',
    app: 'echarts-pie',
    version: '1.0.0',
  },
  {
    type: 'chart:gauge',
    name: '仪表盘',
    category: 'chart',
    pc: 'echarts-gauge',
    app: 'echarts-gauge',
    version: '1.0.0',
  },
  {
    type: 'chart:radar',
    name: '雷达图',
    category: 'chart',
    pc: 'echarts-radar',
    app: 'echarts-radar',
    version: '1.0.0',
  },
  {
    type: 'chart:scatter',
    name: '散点图',
    category: 'chart',
    pc: 'echarts-scatter',
    app: 'echarts-scatter',
    version: '1.0.0',
  },
  {
    type: 'chart:map',
    name: '地图',
    category: 'chart',
    pc: 'echarts-map',
    app: 'echarts-map',
    version: '1.0.0',
  },
},
```

```
// =====
// 装饰组件（DataV 风格）
// =====
```

```
{
  type: 'border:box1',
  name: '装饰边框1',
  category: 'chart',
  pc: 'dv-border-box-1',
  app: 'dv-border-box-1',
  version: '1.0.0',
},
{
  type: 'border:box2',
  name: '装饰边框2',
  category: 'chart',
  pc: 'dv-border-box-2',
```

```
    app: 'dv-border-box-2',
    version: '1.0.0',
  },
  {
    type: 'decoration:1',
    name: '装饰1',
    category: 'chart',
    pc: 'dv-decoration-1',
    app: 'dv-decoration-1',
    version: '1.0.0',
  },
  // =====
  // 数据展示组件
  // =====
  {
    type: 'text:title',
    name: '标题文字',
    category: 'chart',
    pc: 'dv-text',
    app: 'dv-text',
    version: '1.0.0',
  },
  {
    type: 'text:scroll',
    name: '滚动文字',
    category: 'chart',
    pc: 'vue3-marquee',
    app: 'vue3-marquee',
    version: '1.0.0',
  },
  {
    type: 'data:scroll-board',
    name: '滚动列表',
    category: 'chart',
    pc: 'dv-scroll-board',
    app: 'dv-scroll-board',
    version: '1.0.0',
  },
  {
    type: 'data:number',
    name: '数字翻牌',
    category: 'chart',
    pc: 'dv-digital-flop',
    app: 'dv-digital-flop',
    version: '1.0.0',
  },
  // =====
  // 媒体组件
  // =====
  {
    type: 'media:image',
    name: '图片',
    category: 'chart',
    pc: 'img',
    app: 'img',
    version: '1.0.0',
  },
  {
```

```
    type: 'media:video',
    name: '视频',
    category: 'chart',
    pc: 'video',
    app: 'video',
    version: '1.0.0',
  },
  {
    type: 'media:iframe',
    name: '内嵌页面',
    category: 'chart',
    pc: 'iframe',
    app: 'web-view',
    version: '1.0.0',
  },
  // =====
  // 3D 数字孪生组件 (VIP, 标记 version: '2.0.0')
  // =====
  {
    type: '3d:scene',
    name: '3D 场景',
    category: 'chart',
    pc: 'three-scene',
    app: 'three-scene',
    version: '2.0.0',
    // 注释: VIP 功能, 需要 license 验证
  },
  {
    type: '3d:poi',
    name: 'POI 标注',
    category: 'chart',
    pc: 'three-poi',
    app: 'three-poi',
    version: '2.0.0',
  },
  {
    type: '3d:flyline',
    name: '飞线',
    category: 'chart',
    pc: 'three-flyline',
    app: 'three-flyline',
    version: '2.0.0',
  },
  {
    type: '3d:fence',
    name: '电子围栏',
    category: 'chart',
    pc: 'three-fence',
    app: 'three-fence',
    version: '2.0.0',
  },
  {
    type: '3d:heatmap',
    name: '3D 热力图',
    category: 'chart',
    pc: 'three-heatmap',
    app: 'three-heatmap',
    version: '2.0.0',
  },
}
```

```
];
```

F-6a.3 大屏编译器（基础模块）

文件：src/core/compiler/dashboard/compiler.ts

内容：DashboardIR → 可独立运行的 vue3 + ECharts 大屏项目

```
/**
 * 数字大屏编译器
 *
 * 将 DashboardIR 编译为可独立运行的 vue3 + ECharts 大屏项目
 *
 * 生成产物：
 *   src/App.vue           - 入口（自适应缩放）
 *   src/main.ts           - Vue 初始化
 *   src/views/dashboard/index.vue - 大屏主页面
 *   src/components/ChartBar.vue - 柱状图组件
 *   src/components/ChartLine.vue - 折线图组件
 *   src/composables/useChartData.ts - 数据源 Hook
 *   src/composables/useDashboardScale.ts - 缩放适配 Hook
 *   src/config/dashboard.config.json - 原始配置备份
 *   src/styles/theme.css - 主题变量
 *   package.json
 *   vite.config.ts
 *   index.html
 */

import type { DashboardIR } from '../../ir/dashboard-types';
import type { GeneratedProject, CompilerConfig } from '../../vue3/types';

export class DashboardCompiler {
  private config: CompilerConfig;

  constructor(config: Partial<CompilerConfig> & { entity: string }) {
    this.config = {
      entity: config.entity,
      entityLabel: config.entityLabel ?? config.entity,
      apiBasePath: config.apiBasePath ?? `/api/dashboard/${config.entity}`,
      generatorVersion: config.generatorVersion ?? '1.0.0',
    };
  }

  compile(ir: DashboardIR): { project: GeneratedProject; warnings: string[] } {
    const project: GeneratedProject = new Map();
    const warnings: string[] = [];
    const e = this.config.entity;
    const now = new Date().toISOString();
    const version = this.config.generatorVersion;

    // 1. package.json
    project.set('package.json', this.generatePackageJson(ir));

    // 2. vite.config.ts
    project.set('vite.config.ts', this.generateViteConfig());

    // 3. index.html
    project.set('index.html', this.generateIndexHtml(ir));

    // 4. src/main.ts
    project.set('src/main.ts', this.generateMainTs());
  }
}
```

```

// 5. src/App.vue
project.set('src/App.vue', this.generateAppVue(ir));

// 6. src/views/dashboard/index.vue (主页面)
project.set(`src/views/${e}/index.vue`, this.generateDashboardVue(ir));

// 7. 每个 widget 一个组件
for (const widget of ir.widgets) {
  const componentPath = `src/components/${this.widgetToFileName(widget)}`;
  project.set(componentPath, this.generateWidgetComponent(widget, ir));
}

// 8. 数据源 Hook
if (ir.dataSources.length > 0) {
  project.set(`src/composables/useChartData.ts`, this.generateChartDataHook(ir));
}

// 9. 缩放 Hook
project.set(`src/composables/useDashboardScale.ts`, this.generateScaleHook(ir));

// 10. 主题 CSS
project.set(`src/styles/theme.css`, this.generateThemeCss(ir));

// 11. 原始配置备份
project.set(`src/config/${e}.config.json`, JSON.stringify(ir, null, 2));

return { project, warnings };
}

// —— 以下为各文件的生成方法 ——
// 工程师根据 DashboardIR 的字段逐一实现
// 核心思路：
//   图表组件 → vue-echarts + echarts option
//   装饰组件 → @jiaminghi/data-view 组件
//   数据绑定 → useChartData Hook (API/WebSocket/轮询)
//   自适应 → v-scale-screen 或 CSS transform: scale()
//   主题 → CSS variables + 主题 token

private generatePackageJson(ir: DashboardIR): string {
  const now = new Date().toISOString();
  return `{
"name": "jnpf-dashboard-${this.config.entity}",
"version": "1.0.0",
"// @jnpf-generated": "v${this.config.generatorVersion} entity=${this.config.entity}
time=${now}",
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview"
},
"dependencies": {
  "vue": "^3.4.0",
  "echarts": "^5.5.0",
  "vue-echarts": "^7.0.0",
  "@jiaminghi/data-view": "^3.0.0",
  "@vueuse/core": "^11.0.0",
  "vue3-marquee": "^4.0.0",
  "axios": "^1.7.0"
},

```

```

    "devDependencies": {
      "vite": "^5.0.0",
      "@vitejs/plugin-vue": "^5.0.0",
      "typescript": "^5.0.0"
    }
  }`;
}

```

```

  private generateViteConfig(): string {
    return `import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

```

```

export default defineConfig({
  plugins: [vue()],
  server: { port: 3200 }
})`;
}

```

```

  private generateIndexHtml(ir: DashboardIR): string {
    return `<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>${this.config.entityLabel} - 数据大屏</title>
</head>
<body>
  <div id="app"></div>
  <script type="module" src="/src/main.ts"></script>
</body>
</html>`;
  }

```

```

  private generateMainTs(): string {
    return `import { createApp } from 'vue'
import App from './App.vue'
import './styles/theme.css'

```

```

createApp(App).mount('#app')`;
}

```

```

  private generateAppVue(ir: DashboardIR): string {
    return `<template>
<div class="dashboard-app">
  <router-view />
</div>
</template>

```

```

<style>
body { margin: 0; padding: 0; overflow: hidden; background: var(--bg-color, #0d0d0d); }
.dashboard-app { width: 100vw; height: 100vh; }
</style>`;
}

```

```

  private generateDashboardVue(ir: DashboardIR): string {
    const now = new Date().toISOString();
    const widgetImports = ir.widgets.map(w =>
      `import ${this.widgetToComponentName(w)} from
'@/components/${this.widgetToFileName(w)}'`
    ).join('\n');

```



```

const widgetTemplates = ir.widgets.map(w => {
  const pos = w.position;
  return `    <${this.widgetToComponentName(w)}
    style="position: absolute; left: ${pos.x}px; top: ${pos.y}px; width: ${pos.w}px;
height: ${pos.h}px; z-index: ${pos.zIndex ?? 1};"
    />`;
}).join('\n');

return `<!-- @jnpf-generated v${this.config.generatorVersion}
entity=${this.config.entity} type=dashboard -->
<!-- 生成时间: ${now} -->

<template>
  <div class="dashboard" :style="{ width: '${ir.size.width}px', height:
'${ir.size.height}px' }">
    ${widgetTemplates}
  </div>
</template>

<script setup lang="ts">
import { onMounted } from 'vue'
import { useDashboardScale } from '@/composables/useDashboardScale'
${widgetImports}

const { containerRef } = useDashboardScale(${ir.size.width}, ${ir.size.height})

onMounted(() => {
  containerRef.value = document.querySelector('.dashboard') as HTMLElement
})
</script>

<style scoped>
.dashboard {
  position: relative;
  transform-origin: left top;
}
</style>`;
}

// 以下方法工程师实现:
//   generateWidgetComponent(widget, ir) - 根据 widget.type 生成不同组件
//   generateChartDataHook(ir)           - 生成数据源 Hook
//   generateScaleHook(ir)                - 生成缩放 Hook
//   generateThemeCss(ir)                 - 生成主题 CSS
//   widgetToFileName(widget)             - widget → 文件名
//   widgetToComponentName(widget)        - widget → 组件名

private widgetToFileName(w: { type: string; id: string }): string {
  return w.type.replace(/[:/]/g, '-').replace(/([A-Z])/g, '-$1').toLowerCase()
    + '-' + w.id.slice(0, 6) + '.vue';
}

private widgetToComponentName(w: { type: string; id: string }): string {
  const base = w.type.split(':').map(s => s.charAt(0).toUpperCase() +
s.slice(1)).join('');
  return base + w.id.slice(0, 4);
}

private generateWidgetComponent(widget: DashboardWidget, ir: DashboardIR): string {

```

```

// 由工程师根据 widget.type 实现不同的组件模板
// 例如:
//   chart:bar → vue-echarts 柱状图
//   chart:line → vue-echarts 折线图
//   border:box1 → dv-border-box-1
//   text:title → 纯文字
//   3d:scene → Three.js 场景（阶段二实现）
return `<!-- widget: ${widget.type} (${widget.id}) -->
<template>
  <div class="widget widget-${widget.type}">
    <!-- ${widget.type} 组件实现 -->
  </div>
</template>`;
}

private generateChartDataHook(ir: DashboardIR): string {
  // 由工程师实现
  return `// useChartData - 数据源 Hook（待实现）`;
}

private generateScaleHook(ir: DashboardIR): string {
  return `import { ref, onMounted, onUnmounted } from 'vue'

export function useDashboardScale(designwidth: number, designHeight: number) {
  const containerRef = ref<HTMLElement | null>(null)

  function updateScale() {
    if (!containerRef.value) return
    const scaleX = window.innerWidth / designwidth
    const scaleY = window.innerHeight / designHeight
    const scale = Math.min(scaleX, scaleY)
    containerRef.value.style.transform = `scale(${scale})`
  }

  onMounted(() => {
    updateScale()
    window.addEventListener('resize', updateScale)
  })

  onUnmounted(() => {
    window.removeEventListener('resize', updateScale)
  })

  return { containerRef }
}`;
}

private generateThemeCss(ir: DashboardIR): string {
  return `/* @jnpf-generated theme for ${this.config.entity} */
:root {
  --bg-color: #0d0d0d;
  --surface-color: #1a1a1a;
  --primary-color: #00d4ff;
  --text-color: #ffffff;
  --text-muted: #8899aa;
  --border-color: #1e3a5f;
  --success-color: #00e396;
  --warning-color: #feb019;
  --danger-color: #ff4560;
}`;
}

```

```
}  
}
```

F-6a.4 大屏编译器测试

文件: src/core/compiler/__tests__/dashboard-compiler.test.ts

```
import { describe, it, expect } from 'vitest';  
import { DashboardCompiler } from '../dashboard/compiler';  
import type { DashboardIR } from '../ir/dashboard-types';  
  
const mockDashboardIR: DashboardIR = {  
  type: 'dashboard',  
  id: 'test-dashboard',  
  name: '测试大屏',  
  size: { width: 1920, height: 1080 },  
  background: { type: 'color', value: '#0d0d0d' },  
  theme: 'dark',  
  widgets: [  
    {  
      id: 'w1',  
      type: 'chart:bar',  
      position: { x: 50, y: 50, w: 800, h: 400 },  
      props: { title: '月度销售统计' },  
      dataSourceId: 'ds1',  
    },  
    {  
      id: 'w2',  
      type: 'chart:pie',  
      position: { x: 900, y: 50, w: 400, h: 400 },  
      props: { title: '区域分布' },  
      dataSourceId: 'ds1',  
    },  
    {  
      id: 'w3',  
      type: 'border:box1',  
      position: { x: 20, y: 20, w: 1880, h: 1040 },  
      props: {},  
    },  
  ],  
  dataSources: [  
    {  
      id: 'ds1',  
      name: '销售数据',  
      type: 'api',  
      url: '/api/sales/statistics',  
      method: 'GET',  
      pollInterval: 30000,  
    },  
  ],  
};  
  
describe('DashboardCompiler', () => {  
  const compiler = new DashboardCompiler({  
    entity: 'sales-dashboard',  
    entityLabel: '销售大屏',  
  });  
  
  const result = compiler.compile(mockDashboardIR);
```

```

it('生成文件数量正确', () => {
  expect(result.project.size).toBeGreaterThan(5);
});

it('生成 package.json', () => {
  const pkg = result.project.get('package.json')!;
  expect(pkg).toContain('vue-echarts');
  expect(pkg).toContain('@jiaminghi/data-view');
});

it('生成大屏主页面', () => {
  const mainPage = result.project.get('src/views/sales-dashboard/index.vue')!;
  expect(mainPage).toContain('position: absolute');
  expect(mainPage).toContain('1920px');
});

it('每个 widget 生成独立组件', () => {
  for (const widget of mockDashboardIR.widgets) {
    const hasComponent = [...result.project.keys()].some(k =>
      k.startsWith('src/components/') && k.includes(widget.type.replace(':', '-'))
    );
    expect(hasComponent).toBe(true);
  }
});

it('生成缩放 Hook', () => {
  const scaleHook = result.project.get('src/composables/useDashboardScale.ts')!;
  expect(scaleHook).toContain('useDashboardScale');
  expect(scaleHook).toContain('window.innerWidth');
});

it('生成主题 CSS', () => {
  const css = result.project.get('src/styles/theme.css')!;
  expect(css).toContain('--bg-color');
  expect(css).toContain('--primary-color');
});

it('零 eval/Function', () => {
  for (const [, content] of result.project) {
    expect(content).not.toMatch(/\beval\b/);
    expect(content).not.toMatch(/new Function/);
  }
});

it('生成标记存在', () => {
  for (const [path, content] of result.project) {
    if (path.endsWith('.vue') || path.endsWith('.ts')) {
      expect(content).toContain('@jnpf-generated');
    }
  }
});

```

F-6a 交付物

- src/core/ir/dashboard-types.ts – 大屏 IR 类型定义
- src/core/component-registry/builtin-dashboard.ts – 大屏组件注册（含 3D 预留）
- src/core/compiler/dashboard/compiler.ts – 大屏编译器（基础模块）
- src/core/compiler/__tests__/dashboard-compiler.test.ts – 测试
- 标签: v5.2-dashboard-m1

Week 4: 后端清零 Sprint 3 + 阶段一回顾

后端清零并行任务（Sprint 1-3 已完成的部分）：

- Sprint 1: 核心框架模块 App.GetService 消化（约 8 处）
- Sprint 2: 数据库访问层 App.GetService 消化（约 6 处）
- Sprint 3: 认证/授权层 App.GetService 消化（约 5 处）

累计消化：19 / 37 处（剩余 18 处在阶段二继续）

阶段一里程碑验收：

- F-5 端到端验证通过
- F-6a 大屏编译器基础模块通过测试
- 大屏组件注册表完整（图表 + 装饰 + 数据展示 + 媒体 + 3D 预留）
- 后端 Sprint 1-3 完成（19/37 App.GetService 消化）
- 演示项目可独立运行
- ESLint eval/Function 规则生效

以上是阶段一的完整开发计划。工程师请确认后开始执行。说"继续"我贴出阶段二（3D 数字孪生 VIP 模块 + 后端清零 Sprint 4-5）。

阶段二：3D 数字孪生 VIP 模块 + 后端清零（4 周）

目标

实现数字大屏的 3D 数字孪生 VIP 功能模块，
作为独立的高级组件集成到大屏编辑器中，
不影响现有大屏任何基础功能。
后端清零 Sprint 4-5 并行推进。

核心设计原则

- ① 增量开发：3D 是"加模块"，不是"改模块"
 现有图表、装饰、数据展示组件零改动
 3D 组件在组件注册表中已预留（version: '2.0.0'）
- ② VIP 隔离：3D 功能通过 license 控制
 基础版客户看不到 3D 组件
 VIP 客户激活后，3D 组件出现在组件面板中
- ③ 渐进增强：
 week 1: Three.js 基础场景 + 相机 + 光源
 week 2: 模型加载 + 基础数字要素（POI/飞线）
 week 3: 高级要素（围栏/热力）+ 数据绑定
 week 4: 事件绑定 DSL + 代码生成 + 测试

Week 1: Three.js 基础场景

F-6b.1 3D 场景核心组件

文件: src/core/compiler/dashboard/templates/3d/Scene.vue.ejs

内容: Three.js 场景初始化 + 相机 + 光源 + 渲染循环

```
<!-- 3D 场景主组件 -->
<!-- @jnpf-generated v<%= version %> entity=<%= entity %> type=3d-scene -->

<template>
  <div ref="containerRef" class="three-scene" />
</template>

<script setup lang="ts">
import { ref, onMounted, onUnmounted, watch } from 'vue'
import * as THREE from 'three'
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js'

const props = defineProps<{
  /** 场景背景色 */
  backgroundColor?: string
  /** 相机位置 */
  cameraPosition?: [number, number, number]
  /** 相机目标点 */
  cameraTarget?: [number, number, number]
  /** 是否启用轨道控制 */
  enableControls?: boolean
  /** 环境光强度 */
  ambientIntensity?: number
  /** 方向光位置 */
  directionalLightPosition?: [number, number, number]
}>()

const emit = defineEmits<{
  ready: [scene: THREE.Scene, camera: THREE.Camera, renderer: THREE.Renderer]
  click: [intersect: THREE.Intersection | null]
}>()

const containerRef = ref<HTMLElement>()

// Three.js 核心对象
let scene: THREE.Scene
let camera: THREE.PerspectiveCamera
let renderer: THREE.WebGLRenderer
let controls: OrbitControls | null = null
let animationId: number

function initScene() {
  if (!containerRef.value) return

  // 场景
  scene = new THREE.Scene()
  scene.background = new THREE.Color(props.backgroundColor ?? '#0d0d0d')

  // 相机
  const aspect = containerRef.value.clientWidth / containerRef.value.clientHeight
  camera = new THREE.PerspectiveCamera(60, aspect, 0.1, 10000)
  const camPos = props.cameraPosition ?? [10, 10, 10]
```

```

camera.position.set(camPos[0], camPos[1], camPos[2])

const target = props.cameraTarget ?? [0, 0, 0]
camera.lookAt(target[0], target[1], target[2])

// 渲染器
renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true })
renderer.setSize(containerRef.value.clientWidth, containerRef.value.clientHeight)
renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2))
renderer.shadowMap.enabled = true
containerRef.value.appendChild(renderer.domElement)

// 轨道控制
if (props.enableControls !== false) {
  controls = new OrbitControls(camera, renderer.domElement)
  controls.enabledDamping = true
  controls.dampingFactor = 0.05
}

// 环境光
const ambient = new THREE.AmbientLight(0xffffff, props.ambientIntensity ?? 0.6)
scene.add(ambient)

// 方向光
const dirPos = props.directionalLightPosition ?? [50, 50, 50]
const directional = new THREE.DirectionalLight(0xffffff, 0.8)
directional.position.set(dirPos[0], dirPos[1], dirPos[2])
directional.castShadow = true
scene.add(directional)

// 地面辅助（可选）
const gridHelper = new THREE.GridHelper(100, 50, 0x1e3a5f, 0x1e3a5f)
scene.add(gridHelper)

// 渲染循环
function animate() {
  animationId = requestAnimationFrame(animate)
  controls?.update()
  renderer.render(scene, camera)
}
animate()

emit('ready', scene, camera, renderer)
}

// 窗口缩放
function handleResize() {
  if (!containerRef.value) return
  const w = containerRef.value.clientWidth
  const h = containerRef.value.clientHeight
  camera.aspect = w / h
  camera.updateProjectionMatrix()
  renderer.setSize(w, h)
}

// 点击事件（Raycaster）
function handleClick(event: MouseEvent) {
  if (!containerRef.value) return
  const rect = containerRef.value.getBoundingClientRect()
  const mouse = new THREE.Vector2(

```

```

    ((event.clientX - rect.left) / rect.width) * 2 - 1,
    -((event.clientY - rect.top) / rect.height) * 2 + 1
  )
  const raycaster = new THREE.Raycaster()
  raycaster.setFromCamera(mouse, camera)
  const intersects = raycaster.intersectObjects(scene.children, true)
  emit('click', intersects.length > 0 ? intersects[0] : null)
}

onMounted(() => {
  initScene()
  window.addEventListener('resize', handleResize)
  containerRef.value?.addEventListener('click', handleClick)
})

onUnmounted(() => {
  cancelAnimationFrame(animationId)
  window.removeEventListener('resize', handleResize)
  controls?.dispose()
  renderer.dispose()
})

// 暴露给父组件
defineExpose({
  getScene: () => scene,
  getCamera: () => camera,
  getRenderer: () => renderer,
  /** 添加 3D 对象 */
  addObject: (obj: THREE.Object3D) => scene.add(obj),
  /** 移除 3D 对象 */
  removeObject: (obj: THREE.Object3D) => scene.remove(obj),
  /** 根据 ID 查找对象 */
  findObject: (name: string) => scene.getObjectByName(name),
})
</script>

<style scoped>
.three-scene {
  width: 100%;
  height: 100%;
  overflow: hidden;
}
</style>

```

F-6b.2 模型加载器

文件: src/core/compiler/dashboard/templates/3d/ModelLoader.ts

内容: glTF/OBJ/FBX 模型加载 + 缓存

```

/**
 * 3D 模型加载器
 *
 * 支持格式: glTF (推荐)、OBJ、FBX
 * 特点: 加载缓存、进度回调、错误处理
 */

import * as THREE from 'three'
import { GLTFLoader } from 'three/examples/jsm/loaders/GLTFLoader.js'
import { OBJLoader } from 'three/examples/jsm/loaders/OBJLoader.js'
import { DRACOLoader } from 'three/examples/jsm/loaders/DRACOLoader.js'

```



```

// 加载缓存
const cache = new Map<string, THREE.Object3D>()

// Draco 解码器 (glTF 压缩模型)
const dracoLoader = new DRACOLoader()
dracoLoader.setDecoderPath('https://www.gstatic.com/draco/versioned/decoders/1.5.6/')

const gltfLoader = new GLTFLoader()
gltfLoader.setDRACOLoader(dracoLoader)

const objLoader = new OBJLoader()

export interface LoadOptions {
  /** 模型 URL */
  url: string
  /** 模型名称 (用于场景查找) */
  name?: string
  /** 缩放 */
  scale?: [number, number, number]
  /** 位置 */
  position?: [number, number, number]
  /** 旋转 (弧度) */
  rotation?: [number, number, number]
  /** 是否启用阴影 */
  castShadow?: boolean
  /** 进度回调 */
  onProgress?: (loaded: number, total: number) => void
}

/**
 * 加载 3D 模型
 *
 * @returns THREE.Object3D 可直接添加到场景
 */
export async function loadModel(options: LoadOptions): Promise<THREE.Object3D> {
  const { url, name, scale, position, rotation, castShadow, onProgress } = options

  // 检查缓存
  if (cache.has(url)) {
    const cached = cache.get(url)!.clone()
    applyTransforms(cached, { scale, position, rotation, castShadow, name })
    return cached
  }

  // 根据扩展名选择加载器
  const ext = url.split('.').pop()?.toLowerCase()
  let object: THREE.Object3D

  switch (ext) {
    case 'gltf':
    case 'glb': {
      const gltf = await new Promise<any>((resolve, reject) => {
        gltfLoader.load(url, resolve, (e) => onProgress?.(e.loaded, e.total), reject)
      })
      object = gltf.scene
      break
    }
    case 'obj': {
      object = await new Promise<any>((resolve, reject) => {

```

```

        objLoader.load(url, resolve, (e) => onProgress?.(e.loaded, e.total), reject)
    })
    break
}
default:
    throw new Error(`[ModelLoader] 不支持的模型格式: ${ext}`)
}

// 缓存原始模型
cache.set(url, object.clone())

// 应用变换
applyTransforms(object, { scale, position, rotation, castShadow, name })

return object
}

function applyTransforms(
    obj: THREE.Object3D,
    opts: { scale?: number[]; position?: number[]; rotation?: number[]; castShadow?:
boolean; name?: string }
) {
    if (opts.name) obj.name = opts.name
    if (opts.scale) obj.scale.set(opts.scale[0], opts.scale[1], opts.scale[2])
    if (opts.position) obj.position.set(opts.position[0], opts.position[1],
opts.position[2])
    if (opts.rotation) obj.rotation.set(opts.rotation[0], opts.rotation[1],
opts.rotation[2])
    if (opts.castShadow) {
        obj.traverse((child) => {
            if ((child as THREE.Mesh).isMesh) {
                child.castShadow = true
                child.receiveShadow = true
            }
        })
    }
}

/**
 * 清除加载缓存
 */
export function clearModelCache(): void {
    cache.clear()
}

```

Week 2: 数字要素系统

F-6b.3 POI 标注组件

```

/**
 * POI (Point of Interest) 标注
 * 在 3D 场景中标注设备、人员、摄像头等点位
 *
 * 特点:
 *   HTML 标签叠加在 3D 场景上 (CSS2DRenderer)
 *   支持自定义图标和弹窗内容
 *   数据驱动 (从 API 获取 POI 列表)
 */

```

```

import { CSS2DObject } from 'three/examples/jsm/renderers/CSS2DRenderer.js'
import * as THREE from 'three'

export interface POIConfig {
  id: string
  name: string
  /** 3D 坐标 */
  position: [number, number, number]
  /** 图标类型 */
  icon: 'device' | 'camera' | 'person' | 'alarm' | 'custom'
  /** 自定义图标 URL (icon=custom 时使用) */
  iconUrl?: string
  /** 状态: normal / warning / alarm */
  status: 'normal' | 'warning' | 'alarm'
  /** 弹窗内容 */
  popup?: string
  /** 数据绑定 */
  data?: Record<string, unknown>
}

export function createPOI(config: POIConfig): CSS2DObject {
  const div = document.createElement('div')
  div.className = `poi-marker poi-${config.status}`
  div.innerHTML = `
    <div class="poi-icon poi-icon-${config.icon}"></div>
    <div class="poi-label">${config.name}</div>
  `

  // 点击事件
  div.addEventListener('click', () => {
    document.dispatchEvent(new CustomEvent('poi-click', { detail: config }))
  })

  const label = new CSS2DObject(div)
  label.position.set(config.position[0], config.position[1], config.position[2])
  label.name = `poi-${config.id}`

  return label
}

/**
 * 批量创建 POI
 */
export function createPOIGroup(pois: POIConfig[]): THREE.Group {
  const group = new THREE.Group()
  group.name = 'poi-group'
  for (const poi of pois) {
    group.add(createPOI(poi))
  }
  return group
}

```

F-6b.4 飞线组件

```

/**
 * 飞线 (FlyLine)
 * 两点之间的弧形动态飞线效果
 * 常用于: 数据流向、物流路径、信号传输
 */

```

```

import * as THREE from 'three'

export interface FlyLineConfig {
  /** 起点坐标 */
  start: [number, number, number]
  /** 终点坐标 */
  end: [number, number, number]
  /** 飞线颜色 */
  color?: string
  /** 弧线高度（自动计算或手动指定） */
  height?: number
  /** 飞线速度 */
  speed?: number
  /** 飞线宽度 */
  width?: number
}

export function createFlyLine(config: FlyLineConfig): THREE.Group {
  const group = new THREE.Group()
  const start = new THREE.Vector3(...config.start)
  const end = new THREE.Vector3(...config.end)

  // 计算弧线控制点
  const mid = new THREE.Vector3().addVectors(start, end).multiplyScalar(0.5)
  const dist = start.distanceTo(end)
  mid.y += config.height ?? dist * 0.4

  // 创建弧线曲线
  const curve = new THREE.QuadraticBezierCurve3(start, mid, end)
  const points = curve.getPoints(64)
  const geometry = new THREE.BufferGeometry().setFromPoints(points)

  // 飞线材质（渐变 + 动态）
  const material = new THREE.LineBasicMaterial({
    color: config.color ?? '#00d4ff',
    transparent: true,
    opacity: 0.8,
  })

  const line = new THREE.Line(geometry, material)
  line.name = 'flyline-base'
  group.add(line)

  // 飞行粒子
  const particleGeom = new THREE.SphereGeometry(config.width ?? 0.1, 8, 8)
  const particleMat = new THREE.MeshBasicMaterial({
    color: config.color ?? '#00d4ff',
    transparent: true,
    opacity: 1,
  })
  const particle = new THREE.Mesh(particleGeom, particleMat)
  particle.name = 'flyline-particle'
  group.add(particle)

  // 动画
  const speed = config.speed ?? 0.005
  let t = 0
  group.userData.animate = () => {
    t = (t + speed) % 1
  }
}

```

```

        const point = curve.getPoint(t)
        particle.position.copy(point)
    }

    return group
}

/**
 * 批量创建飞线
 */
export function createFlyLineGroup(configs: FlyLineConfig[]): THREE.Group {
    const group = new THREE.Group()
    group.name = 'flyline-group'
    for (const config of configs) {
        group.add(createFlyLine(config))
    }
    return group
}

```

Week 3: 围栏 + 热力图 + 数据绑定

F-6b.5 电子围栏

```

/**
 * 电子围栏（Fence）
 * 在 3D 场景中绘制区域边界
 * 常用于：安防区域、施工禁区、地理围栏告警
 */

import * as THREE from 'three'

export interface FenceConfig {
    id: string
    name: string
    /** 围栏顶点坐标（闭合多边形，至少 3 个点） */
    points: [number, number, number][]
    /** 围栏高度 */
    height?: number
    /** 正常颜色 */
    color?: string
    /** 告警颜色 */
    alarmColor?: string
    /** 当前状态 */
    status: 'normal' | 'alarm'
    /** 透明度 */
    opacity?: number
}

export function createFence(config: FenceConfig): THREE.Group {
    const group = new THREE.Group()
    group.name = `fence-${config.id}`

    const height = config.height ?? 3
    const color = config.status === 'alarm'
        ? (config.alarmColor ?? '#ff4560')
        : (config.color ?? '#00d4ff')
    const opacity = config.opacity ?? 0.3

```

```

// 地面区域（半透明填充）
const shape = new THREE.Shape()
const groundPoints = config.points.map(p => new THREE.Vector2(p[0], p[2]))
shape.moveTo(groundPoints[0].x, groundPoints[0].y)
for (let i = 1; i < groundPoints.length; i++) {
  shape.lineTo(groundPoints[i].x, groundPoints[i].y)
}
shape.closePath()

const groundGeom = new THREE.ShapeGeometry(shape)
const groundMat = new THREE.MeshBasicMaterial({
  color,
  transparent: true,
  opacity: opacity * 0.5,
  side: THREE.DoubleSide,
})
const ground = new THREE.Mesh(groundGeom, groundMat)
ground.rotation.x = -Math.PI / 2
ground.position.y = config.points[0][1] ?? 0
group.add(ground)

// 围栏墙壁
for (let i = 0; i < config.points.length; i++) {
  const p1 = config.points[i]
  const p2 = config.points[(i + 1) % config.points.length]

  const wallGeom = new THREE.PlaneGeometry(
    Math.sqrt((p2[0] - p1[0]) ** 2 + (p2[2] - p1[2]) ** 2),
    height
  )
  const wallMat = new THREE.MeshBasicMaterial({
    color,
    transparent: true,
    opacity,
    side: THREE.DoubleSide,
  })
  const wall = new THREE.Mesh(wallGeom, wallMat)

  // 定位和旋转
  const midX = (p1[0] + p2[0]) / 2
  const midZ = (p1[2] + p2[2]) / 2
  wall.position.set(midX, (p1[1] ?? 0) + height / 2, midZ)
  wall.rotation.y = Math.atan2(p2[2] - p1[2], p2[0] - p1[0])

  group.add(wall)
}

// 存储配置（用于数据驱动更新）
group.userData.fenceConfig = config

return group
}

/**
 * 更新围栏状态
 */
export function updateFenceStatus(fence: THREE.Group, status: 'normal' | 'alarm') {
  const config = fence.userData.fenceConfig as FenceConfig
  config.status = status
  const color = status === 'alarm'

```

```

    ? (config.alarmColor ?? '#ff4560')
    : (config.color ?? '#00d4ff')

fence.traverse((child) => {
  if ((child as THREE.Mesh).isMesh) {
    const mat = (child as THREE.Mesh).material as THREE.MeshBasicMaterial
    mat.color.set(color)
  }
})
}

```

F-6b.6 3D 热力图

```

/**
 * 3D 热力图
 * 在 3D 场景中以柱状/平面热力形式展示数据密度
 * 常用于：人流密度、设备分布、告警集中区域
 */

import * as THREE from 'three'

export interface HeatmapPoint {
  /** 坐标 */
  position: [number, number, number]
  /** 值（0-1 归一化） */
  value: number
  /** 标签 */
  label?: string
}

export interface HeatmapConfig {
  id: string
  points: HeatmapPoint[]
  /** 最大高度 */
  maxHeight?: number
  /** 最小高度 */
  minHeight?: number
  /** 低温颜色 */
  coldColor?: string
  /** 高温颜色 */
  hotColor?: string
  /** 是否为柱状模式（false 则为平面热力） */
  barMode?: boolean
  /** 柱状半径 */
  barRadius?: number
}

export function createHeatmap(config: HeatmapConfig): THREE.Group {
  const group = new THREE.Group()
  group.name = `heatmap-${config.id}`

  const maxH = config.maxHeight ?? 10
  const minH = config.minHeight ?? 0.1
  const coldColor = new THREE.Color(config.coldColor ?? '#00d4ff')
  const hotColor = new THREE.Color(config.hotColor ?? '#ff4560')

  for (const point of config.points) {
    const height = minH + (maxH - minH) * point.value
    const color = coldColor.clone().lerp(hotColor, point.value)
  }
}

```

```

if (config.barMode !== false) {
  // 柱状模式
  const radius = config.barRadius ?? 0.3
  const geom = new THREE.CylinderGeometry(radius, radius, height, 16)
  const mat = new THREE.MeshBasicMaterial({
    color,
    transparent: true,
    opacity: 0.7,
  })
  const bar = new THREE.Mesh(geom, mat)
  bar.position.set(point.position[0], height / 2, point.position[2])
  group.add(bar)
} else {
  // 平面热力模式
  const radius = 2
  const geom = new THREE.CircleGeometry(radius, 32)
  const mat = new THREE.MeshBasicMaterial({
    color,
    transparent: true,
    opacity: 0.3 + point.value * 0.4,
    side: THREE.Doubleside,
  })
  const circle = new THREE.Mesh(geom, mat)
  circle.rotation.x = -Math.PI / 2
  circle.position.set(point.position[0], 0.01, point.position[2])
  group.add(circle)
}
}

return group
}

```

F-6b.7 数据绑定层

```

/**
 * 3D 数据绑定层
 *
 * 将 API/WebSocket 数据实时映射到 3D 场景中的要素
 *
 * 核心能力：
 * 1. POI 状态实时更新（设备状态变化 → 图标/颜色变化）
 * 2. 飞线数据流驱动（数据量 → 飞线密度/速度）
 * 3. 围栏告警联动（越界事件 → 围栏变红）
 * 4. 热力图数据刷新（人流/设备密度 → 热力分布）
 */

```

```

import type { POIConfig } from './POI'
import type { FenceConfig } from './Fence'
import type { HeatmapPoint } from './Heatmap'

export interface DataBindingRule {
  /** 绑定的目标要素 ID */
  targetId: string
  /** 目标要素类型 */
  targetType: 'poi' | 'fence' | 'heatmap' | 'flyline'
  /** 数据字段路径（如 'data.temperature'） */
  dataField: string
  /** 映射规则 */
  mapping: {
    /** 值 → 状态 */
  }
}

```



```

        condition: string    // 如 "> 80" 或 "== 'offline'"
        action: string       // 如 "status=alarm" 或 "color=red"
    }[]
}

/**
 * 应用数据绑定规则
 * 当数据更新时，根据规则更新 3D 场景中的要素
 */
export function applyDataBindings(
    rules: DataBindingRule[],
    data: Record<string, unknown>,
    scene: { findObject: (name: string) => THREE.Object3D | undefined }
) {
    for (const rule of rules) {
        const obj = scene.findObject(`${rule.targetType}-${rule.targetId}`)
        if (!obj) continue

        const value = getNestedValue(data, rule.dataField)
        if (value === undefined) continue

        for (const mapping of rule.mapping) {
            if (evaluateCondition(value, mapping.condition)) {
                applyAction(obj, mapping.action)
                break
            }
        }
    }
}

function getNestedValue(obj: unknown, path: string): unknown {
    return path.split('.').reduce((o, k) => (o as Record<string, unknown>)?.[k], obj)
}

function evaluateCondition(value: unknown, condition: string): boolean {
    // 简单条件求值（安全，不用 eval）
    const match = condition.match(/^(>|<|=|!=)\s*(.+)$/);
    if (!match) return String(value) === condition

    const op = match[1]
    const target = isNaN(Number(match[2])) ? match[2].replace(/['"]/g, '') :
    Number(match[2])
    const numVal = Number(value)

    switch (op) {
        case '>': return numVal > (target as number)
        case '>=': return numVal >= (target as number)
        case '<': return numVal < (target as number)
        case '<=': return numVal <= (target as number)
        case '==': return value == target
        case '!=': return value != target
        default: return false
    }
}

function applyAction(obj: THREE.Object3D, action: string) {
    const [key, val] = action.split('=')
    switch (key) {
        case 'status':
            obj.userData.status = val
    }
}

```

```

        break
    case 'color':
        obj.traverse((child) => {
            if ((child as THREE.Mesh).isMesh) {
                ((child as THREE.Mesh).material as THREE.MeshBasicMaterial).color.set(val)
            }
        })
        break
    case 'visible':
        obj.visible = val === 'true'
        break
    }
}

```

Week 4: 事件绑定 DSL + 代码生成 + 测试

F-6b.8 大屏事件绑定 DSL（并入表达式引擎，v5.0）

v5.0 裁定：废止独立「蓝图逻辑引擎」模块名；事件→条件→动作链 并入 src/core/expression/engine 的事件绑定 DSL（与 F-2 表达式引擎共用求值器）。围栏/热力/数据绑定仍在阶段二 **4 周全量**交付；PoC-B 未通过时仅允许性能层 LOD 优化。

```

/**
 * 蓝图逻辑引擎（链式配置形态，非 UE 级可视化编辑器）
 *
 * 设计理念：
 *   不实现完整的蓝图编辑器（太重）
 *   实现"事件 → 条件 → 动作"的链式配置
 *   在代码生成时转化为普通的 TypeScript 事件处理代码
 */

export interface BlueprintNode {
    id: string;
    type: 'event' | 'condition' | 'action';
    /** 节点配置 */
    config: Record<string, unknown>;
    /** 下一个节点 ID */
    next?: string;
    /** 条件为 true 时的下一个节点 ID */
    nextTrue?: string;
    /** 条件为 false 时的下一个节点 ID */
    nextFalse?: string;
}

export interface BlueprintFlow {
    id: string;
    name: string;
    nodes: BlueprintNode[];
}

/**
 * 将蓝图流程编译为 TypeScript 事件处理代码
 */
export function compileBlueprint(flow: BlueprintFlow): string {
    const lines: string[] = [];
    lines.push(`// Blueprint: ${flow.name}`);
    lines.push(`// @jnpf-generated blueprint`);
    lines.push('');

```

```

// 找到事件入口节点
const eventNodes = flow.nodes.filter(n => n.type === 'event');

for (const eventNode of eventNodes) {
  const trigger = eventNode.config.trigger as string; // 'click' | 'hover' | 'data-
change'
  const target = eventNode.config.target as string; // 目标要素 ID

  lines.push(`// 事件: ${trigger} on ${target}`);
  lines.push(`function handle_${eventNode.id}(event) {`);

  // 遍历后续节点
  let currentNodeId: string | undefined = eventNode.next;
  while (currentNodeId) {
    const node = flow.nodes.find(n => n.id === currentNodeId);
    if (!node) break;

    switch (node.type) {
      case 'condition': {
        const field = node.config.field as string;
        const op = node.config.operator as string;
        const value = node.config.value as string;
        lines.push(`  if (evaluateCondition(data.${field}, '${op}', '${value}')) {`);
        currentNodeId = node.nextTrue;
        break;
      }
      case 'action': {
        const actionType = node.config.type as string;
        const targetId = node.config.target as string;
        switch (actionType) {
          case 'highlight':
            lines.push(`    highlightElement('${targetId}');`);
            break;
          case 'navigate':
            lines.push(`    navigateTo('${node.config.url}');`);
            break;
          case 'show-popup':
            lines.push(`    showPopup('${targetId}',
${JSON.stringify(node.config.popupData)});`);
            break;
          case 'update-data':
            lines.push(`    updateDataSource('${targetId}');`);
            break;
        }
        currentNodeId = node.next;
        break;
      }
    }
  }
}

lines.push('}');
lines.push('');
}

return lines.join('\n');
}

```

F-6b.9 3D 编译器集成

在大屏编译器中集成 3D 组件的代码生成：

修改文件：src/core/compiler/dashboard/compiler.ts

新增方法：generate3DScene(widget, ir) - 当 widget.type 以 '3d:' 开头时调用

// 在 DashboardCompiler 中新增

```
private generate3DSceneComponent(widget: DashboardWidget, ir: DashboardIR): string {
  const now = new Date().toISOString();
  const version = this.config.generatorVersion;

  return `<!-- @jnpf-generated v${version} type=3d-scene -->
<!-- 生成时间: ${now} -->

<template>
  <ThreeScene
    ref="sceneRef"
    :background-color="${ir.background.value}"
    :camera-position="${JSON.stringify(widget.props.cameraPosition ?? [10, 10, 10])}"
    :enable-controls="${widget.props.enableControls !== false}"
    @ready="onSceneReady"
  />
</template>

<script setup lang="ts">
import { ref } from 'vue'
import ThreeScene from '@components/3d/Scene.vue'
import { loadModel } from '@utils/3d/ModelLoader'
import { createPOIGroup } from '@utils/3d/POI'
import { createFlyLineGroup } from '@utils/3d/FlyLine'
import { createFence } from '@utils/3d/Fence'
import { createHeatmap } from '@utils/3d/Heatmap'
import type { POIConfig } from '@utils/3d/POI'
import type { FlyLineConfig } from '@utils/3d/FlyLine'
import type { FenceConfig } from '@utils/3d/Fence'
import type { HeatmapConfig } from '@utils/3d/Heatmap'

const sceneRef = ref()

// 数据
const pois: POIConfig[] = ${JSON.stringify(widget.props.pois ?? [], null, 2)}
const flylines: FlyLineConfig[] = ${JSON.stringify(widget.props.flylines ?? [], null, 2)}
const fences: FenceConfig[] = ${JSON.stringify(widget.props.fences ?? [], null, 2)}
const heatmap: HeatmapConfig = ${JSON.stringify(widget.props.heatmap ?? { id: 'h1', points: [] }, null, 2)}

async function onSceneReady(scene, camera, renderer) {
  // 加载模型
  const modelUrl = '${widget.props.modelUrl ?? ''}'
  if (modelUrl) {
    const model = await loadModel({
      url: modelUrl,
      name: 'main-model',
      scale: ${JSON.stringify(widget.props.modelScale ?? [1, 1, 1])},
    })
    scene.add(model)
  }
}
```

```

// 添加 POI
if (pois.length > 0) {
  scene.add(createPOIGroup(pois))
}

// 添加飞线
if (flylines.length > 0) {
  const flylineGroup = createFlyLineGroup(flylines)
  scene.add(flylineGroup)
}

// 添加围栏
for (const fenceConfig of fences) {
  scene.add(createFence(fenceConfig))
}

// 添加热力图
if (heatmap.points.length > 0) {
  scene.add(createHeatmap(heatmap))
}
}
</script>`;
}

```

F-6b.10 测试

文件: src/core/compiler/__tests__/dashboard-3d.test.ts

```

import { describe, it, expect } from 'vitest';
import { DashboardCompiler } from '../dashboard/compiler';
import type { DashboardIR } from '../ir/dashboard-types';

const mock3DDashboard: DashboardIR = {
  type: 'dashboard',
  id: 'smart-site-3d',
  name: '智慧工地 3D 大屏',
  size: { width: 1920, height: 1080 },
  background: { type: 'color', value: '#0a0a1a' },
  theme: 'dark-tech',
  widgets: [
    {
      id: 'scene1',
      type: '3d:scene',
      position: { x: 0, y: 0, w: 1920, h: 1080, zIndex: 0 },
      props: {
        modelUrl: '/models/construction-site.glb',
        modelScale: [0.01, 0.01, 0.01],
        cameraPosition: [50, 30, 50],
        pois: [
          { id: 'p1', name: '塔吊A', position: [10, 15, 5], icon: 'device', status:
'normal' },
          { id: 'p2', name: '摄像头B', position: [-5, 8, 10], icon: 'camera', status:
'normal' },
        ],
        flylines: [
          { start: [0, 5, 0], end: [20, 5, 10], color: '#00ff88' },
        ],
        fences: [

```

```

        { id: 'f1', name: '施工禁区', points: [[0, 0, 0], [10, 0, 0], [10, 0, 10], [0,
0, 10]], status: 'normal' },
      ],
    },
    dataSourceId: 'iot-data',
  },
  {
    id: 'chart1',
    type: 'chart:bar',
    position: { x: 50, y: 50, w: 500, h: 300, zIndex: 10 },
    props: { title: '实时告警统计' },
    dataSourceId: 'alarm-data',
  },
  {
    id: 'border1',
    type: 'border:box1',
    position: { x: 10, y: 10, w: 600, h: 350, zIndex: 5 },
    props: {},
  },
],
dataSources: [
  { id: 'iot-data', name: 'IoT 数据', type: 'websocket', url:
'ws://localhost:8080/iot' },
  { id: 'alarm-data', name: '告警数据', type: 'api', url: '/api/alarms/statistics',
pollInterval: 10000 },
],
};

```

```

describe('DashboardCompiler - 3D Digital Twin', () => {
  const compiler = new DashboardCompiler({
    entity: 'smart-site-3d',
    entityLabel: '智慧工地 3D',
  });

  const result = compiler.compile(mock3DDashboard);

  it('生成 3D 场景组件', () => {
    const scene3d = [...result.project.entries()].find(
      ([path]) => path.includes('components/') && path.includes('3d')
    );
    expect(scene3d).toBeDefined();
    expect(scene3d![1]).toContain('ThreeScene');
    expect(scene3d![1]).toContain('loadModel');
    expect(scene3d![1]).toContain('createPOIGroup');
    expect(scene3d![1]).toContain('createFlyLineGroup');
    expect(scene3d![1]).toContain('createFence');
    expect(scene3d![1]).toContain('createHeatmap');
  });

  it('3D 场景不与普通图表冲突', () => {
    // 普通图表组件仍然正常生成
    const chartBar = [...result.project.entries()].find(
      ([path]) => path.includes('components/') && path.includes('chart-bar')
    );
    expect(chartBar).toBeDefined();
  });

  it('数据源 websocket 生成正确', () => {
    const dataHook = result.project.get('src/composables/useChartData.ts');
    // 数据源中包含 websocket, hook 应支持
  });

```

```

    // 由工程师实现时验证
  });

  it('package.json 包含 Three.js 依赖', () => {
    const pkg = result.project.get('package.json')!;
    expect(pkg).toContain('three');
  });

  it('零 eval/Function', () => {
    for (const [, content] of result.project) {
      expect(content).not.toMatch(/\beval\b/);
      expect(content).not.toMatch(/new Function/);
    }
  });
});
});

```

F-6b 交付物

- src/core/compiler/dashboard/templates/3d/Scene.vue.ejs – 3D 场景组件
- src/core/compiler/dashboard/templates/3d/ModelLoader.ts – 模型加载器
- src/core/compiler/dashboard/templates/3d/POI.ts – POI 标注
- src/core/compiler/dashboard/templates/3d/FlyLine.ts – 飞线
- src/core/compiler/dashboard/templates/3d/Fence.ts – 电子围栏
- src/core/compiler/dashboard/templates/3d/Heatmap.ts – 3D 热力图
- src/core/compiler/dashboard/templates/3d/DataBinding.ts – 数据绑定层
- src/core/compiler/dashboard/templates/3d/Blueprint.ts – 蓝图逻辑引擎
- compiler.ts 修改（集成 3D 组件生成）
- src/core/compiler/__tests__/dashboard-3d.test.ts – 3D 测试
- 标签：v5.2-dashbaord-3d-m1

Week 4 附加：后端清零 Sprint 4-5

后端清零并行任务：

Sprint 4: 工作流引擎 App.GetService 消化（约 6 处）

Sprint 5: 消息/通知模块 App.GetService 消化（约 5 处）

累计消化：30 / 37 处（剩余 7 处在阶段三继续）

同时推进 CreateScope 清零：

Sprint 4-5: CreateScope 24 处开始消化（约 10 处）

剩余 14 处在阶段三继续

阶段二里程碑验收

- F-6b 3D 数字孪生基础组件全部通过测试
- 3D 场景编译器集成成功（能生成包含 3D 组件的大屏项目）
- POI/飞线/围栏/热力图 数字要素系统完整
- 蓝图逻辑引擎可生成事件处理代码
- 3D 功能不影响现有大屏基础模块
- 后端 Sprint 4-5 完成（30/37 App.GetService + 10/24 CreateScope）
- package.json 包含 three.js 依赖
- 标签：v5.2-dashbaord-3d-m1

阶段二完成。说“继续”我贴出阶段三（UniApp 架构重构 + 代码生成 + 后端清零收尾）。

阶段三：UniApp 单轨编译器 + FlowIR + 后端清零收尾（4 周）

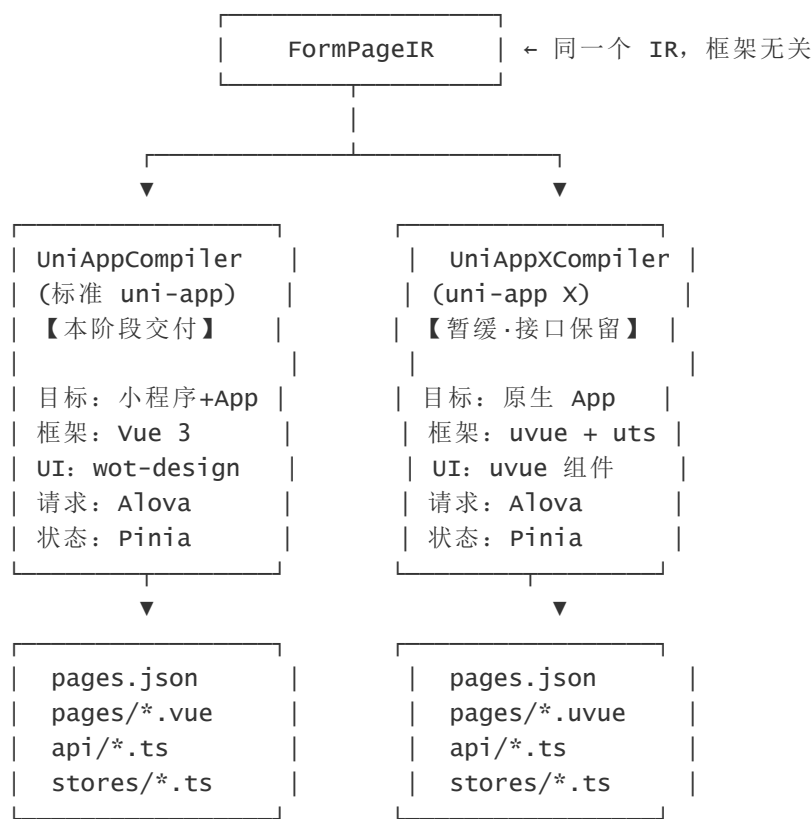
v5.0 裁定 (创始人)：暂缓 uni-app X 双轨，非删除。阶段三交付 标准 uni-app 单轨（小程序 + App）；UniAppXCompiler 接口与 IR 扩展位保留，待 uvue 生态成熟后再启 PoC-A / F-7.5。

目标

实现 UniApp 代码生成器（单轨制）：
模式 A：标准 uni-app（vue 3）→ 小程序端 + App 端（微信/支付宝/抖音/H5）
模式 B：uni-app X（uvue + uts）→ 【暂缓】保留 IR/编译器接口，不纳入本阶段验收

并行交付 FlowIR v1（F-7.9）与后端清零收尾（业务层 App.GetService 口径，非 Furion 框架基因）。

架构设计（单轨交付 + 双轨扩展位）



Week 1：UniApp 共用层 + 请求库升级

F-7.1 Alova 请求封装

```
/**
 * UniApp 通用请求封装（Alova）
 * v3.0：对齐 JNPF RESTfulResult + PC 端 axios 拦截器（jnpf-web-
vue3/src/enums/httpEnum.ts）
 */
import { createAlova } from 'alova';
import AdapterUniapp from '@alova/adapter-uniapp';
```



```

import { apiBaseUrl, getToken, clearToken } from './config';

/** 与 src/enums/httpEnum.ts ResultEnum 一致 */
const ResultCode = {
  SUCCESS: 200,
  SUCCESS_ALT: 0,
  TOKEN_TIMEOUT: 600,
  TOKEN_LOGGED: 601,
  TOKEN_ERROR: 602,
} as const;

function isTokenBusinessCode(code: number): boolean {
  return (
    code === ResultCode.TOKEN_TIMEOUT ||
    code === ResultCode.TOKEN_LOGGED ||
    code === ResultCode.TOKEN_ERROR
  );
}

function handleTokenExpired(): void {
  clearToken();
  uni.reLaunch({ url: '/pages/login/login' });
}

export const alova = createAlova({
  baseUrl: apiBaseUrl,

  beforeRequest(method) {
    const token = getToken();
    if (token) {
      method.config.headers = {
        ...method.config.headers,
        Authorization: `Bearer ${token}`,
      };
    }
  },

  responded: {
    onSuccess: async (response) => {
      const data = response.data as Record<string, unknown>;
      const code = data.code as number;

      if (code === ResultCode.SUCCESS || code === ResultCode.SUCCESS_ALT) {
        return data.data;
      }

      if (isTokenBusinessCode(code)) {
        handleTokenExpired();
        throw new Error(`${data.msg as string} || '登录已过期'`);
      }

      throw new Error(`${data.msg as string} || '请求失败'`);
    },

    onError: (error) => {
      // HTTP 401 层 (非 RESTfulResult body)
      const status = (error as { status?: number }).status;
      if (status === 401) {
        handleTokenExpired();
      }
    }
  }
});

```

```

        console.error('[Alova] 请求错误:', error);
        throw error;
    },
},
...AdapterUniapp(),
});

/**
 * JNPF API 基础方法封装
 * 所有实体 API 都继承这套基础方法
 */

import { alova } from './request';

/**
 * 创建实体 CRUD API 基础方法
 * 复用于所有实体（学生、订单、设备...）
 */
export function createEntityApi<T>(basePath: string) {
    return {
        /** 列表查询 */
        list(params: Record<string, unknown>) {
            return alova.Get<T[]>(`${basePath}/list`, {
                params,
                cache: { mode: 'stale-while-revalidate', expire: 30_000 },
            });
        },

        /** 详情 */
        detail(id: string) {
            return alova.Get<T>(`${basePath}/${id}`);
        },

        /** 新增 */
        create(data: Partial<T>) {
            return alova.Post<T>(basePath, data);
        },

        /** 更新 */
        update(id: string, data: Partial<T>) {
            return alova.Put(`${basePath}/${id}`, data);
        },

        /** 删除 */
        delete(id: string) {
            return alova.Delete(`${basePath}/${id}`);
        },

        /** 批量删除 */
        batchDelete(ids: string[]) {
            return alova.Delete(`${basePath}/batch`, { data: { ids } });
        },
    };
}

```

F-7.2 Pinia Store 基础模板

```
/**
 * UniApp 通用 Store 模板
 * 用于生成每个实体的状态管理
 */

import { defineStore } from 'pinia';
import { ref, reactive } from 'vue';

export function createEntityStore<T extends { id?: string }>(name: string, api:
ReturnType<typeof createEntityApi>) {
  return defineStore(name, () => {
    const loading = ref(false);
    const list = ref<T[]>([]);
    const current = ref<T | undefined>();
    const pagination = reactive({ current: 1, pageSize: 20, total: 0 });
    const searchParams = reactive<Record<string, unknown>>({});

    async function loadList() {
      loading.value = true;
      try {
        const data = await api.list({
          currentPage: pagination.current,
          pageSize: pagination.pageSize,
          ...searchParams,
        });
        list.value = data as T[];
      } finally {
        loading.value = false;
      }
    }

    async function loadDetail(id: string) {
      loading.value = true;
      try {
        current.value = (await api.detail(id)) as T;
      } finally {
        loading.value = false;
      }
    }

    async function save(data: Partial<T>) {
      if (current.value?.id) {
        await api.update(current.value.id, data);
      } else {
        await api.create(data);
      }
    }

    async function remove(id: string) {
      await api.delete(id);
      await loadList();
    }

    return {
      loading, list, current, pagination, searchParams,
      loadList, loadDetail, save, remove,
    };
  });
}
```

```
});  
}
```

F-7.3 小程序端页面模板（标准 uni-app）

```
<!-- pages/entity/list.vue（小程序端列表页模板） -->  
<!-- @jnpf-generated v<%= version %> entity=<%= entity %> platform=mp-weixin -->  
  
<template>  
  <view class="page-list">  
    <!-- 搜索栏 -->  
    <view class="search-bar">  
<% searchFields.forEach(function(sf) { %>  
      <wd-input  
        v-model="searchParams.<%= sf.field %>"  
        placeholder="请输入<%= sf.label %>"  
        clearable  
      />  
<% }); %>  
    <view class="search-actions">  
      <wd-button type="primary" size="small" @click="handleSearch">查询</wd-button>  
      <wd-button size="small" @click="handleReset">重置</wd-button>  
    </view>  
  </view>  
  
  <!-- 列表 -->  
  <wd-cell-group>  
    <wd-cell  
      v-for="item in store.list"  
      :key="item.id"  
      :title="item.<%= displayField %>"  
      @click="handleDetail(item)"  
    >  
      <template #value>  
        <view class="cell-actions">  
          <wd-button size="mini" @click.stop="handleEdit(item)">编辑</wd-button>  
          <wd-button size="mini" type="error" @click.stop="handleDelete(item)">删除  
</wd-button>  
        </view>  
      </template>  
    </wd-cell>  
  </wd-cell-group>  
  
  <!-- 空状态 -->  
  <wd-status-tip v-if="!store.loading && store.list.length === 0" tip="暂无数据" />  
  
  <!-- 加载状态 -->  
  <wd-loading v-if="store.loading" />  
  
  <!-- 悬浮新增按钮 -->  
  <view class="fab" @click="handleAdd">  
    <wd-icon name="add" size="24px" />  
  </view>  
</view>  
</template>  
  
<script setup lang="ts">  
import { onMounted, reactive } from 'vue';  
import { onPullDownRefresh, onReachBottom } from '@dcloudio/uni-app';
```

```

// Store (由编译器生成具体实现)
const store = use<%= Entity %>Store();

const searchParams = reactive<Record<string, string>>({
  <%= searchFields.forEach(function(sf) { %>
    <%= sf.field %>: '',
  <% }); %>
});

onMounted(() => {
  store.loadList();
});

// 下拉刷新
onPullDownRefresh(async () => {
  store.pagination.current = 1;
  await store.loadList();
  uni.stopPullDownRefresh();
});

// 上拉加载更多
onReachBottom(() => {
  store.pagination.current++;
  store.loadList();
});

function handleSearch() {
  Object.assign(store.searchParams, searchParams);
  store.pagination.current = 1;
  store.loadList();
}

function handleReset() {
  Object.keys(searchParams).forEach(k => searchParams[k] = '');
  handleSearch();
}

function handleAdd() {
  uni.navigateTo({ url: '/pages/<%= entity %>/form' });
}

function handleEdit(item) {
  uni.navigateTo({ url: `/pages/<%= entity %>/form?id=${item.id}` });
}

function handleDetail(item) {
  uni.navigateTo({ url: `/pages/<%= entity %>/detail?id=${item.id}` });
}

async function handleDelete(item) {
  uni.showModal({
    title: '确认删除',
    content: `确定删除 ${item.<%= displayField %>}?`,
    success: async (res) => {
      if (res.confirm) {
        await store.remove(item.id);
        uni.showToast({ title: '删除成功' });
      }
    },
  });
}

```

```

}
</script>

<style scoped lang="scss">
.page-list {
  padding: 24rpx;
  padding-bottom: 120rpx;
}
.search-bar {
  margin-bottom: 24rpx;
}
.search-actions {
  display: flex;
  gap: 16rpx;
  margin-top: 16rpx;
}
.fab {
  position: fixed;
  right: 40rpx;
  bottom: 120rpx;
  width: 100rpx;
  height: 100rpx;
  border-radius: 50%;
  background: #0083ff;
  display: flex;
  align-items: center;
  justify-content: center;
  color: #fff;
  box-shadow: 0 4rpx 16rpx rgba(0, 0, 0, 0.2);
}
</style>

<!-- pages/entity/form.vue (小程序端表单页模板) -->
<!-- @jnpf-generated v<%= version %> entity=<%= entity %> platform=mp-weixin -->

<template>
  <view class="page-form">
    <wd-form ref="formRef" :model="formData">
<% fields.forEach(function(field) { %>
    <!-- <%= field.label %> -->
    <wd-cell title="<%= field.label %>" required="<%= field.config.required %>">
<% if (field.component.app === 'uni-easyinput') { %>
      <wd-input
        v-model="formData.<%= field.model %>"
        placeholder="请输入<%= field.label %>"
<% if (field.config.required) { %>
        required
<% } %>
    />
<% } else if (field.component.app === 'uni-data-select') { %>
      <wd-select-picker
        v-model="formData.<%= field.model %>"
        :columns="<%= field.model %>_options"
        placeholder="请选择<%= field.label %>"
      />
<% } else if (field.component.app === 'uni-datetime-picker') { %>
      <wd-datetime-picker
        v-model="formData.<%= field.model %>"
        placeholder="请选择<%= field.label %>"
      />

```

```

<% } else if (field.component.app === 'switch') { %>
    <wd-switch v-model="formData.<%= field.model %>" />
<% } else { %>
    <wd-input
        v-model="formData.<%= field.model %>"
        placeholder="请输入<%= field.label %>"
    />
<% } %>
</wd-cell>
<% }); %>
</wd-form>

<!-- @jnpf-gen:insert-point=custom-form-fields -->
<!-- @jnpf-gen:end-insert-point=custom-form-fields -->

<view class="form-actions">
    <wd-button type="primary" block @click="handleSubmit">提交</wd-button>
    <wd-button block @click="handleCancel" style="margin-top: 16px">取消</wd-button>
</view>
</view>
</template>

<script setup lang="ts">
import { ref, reactive, onMounted } from 'vue';

const formRef = ref();
const isEdit = ref(false);

const formData = reactive<Record<string, unknown>>>({
    <% fields.forEach(function(field) { %>
        <%= field.model %>: <%- getDefaultValue(field) %>,
    <% }); %>
});

onMounted(async () => {
    const pages = getCurrentPages();
    const page = pages[pages.length - 1];
    const id = page?.options?.id;
    if (id) {
        isEdit.value = true;
        const detail = await api.detail(id);
        Object.assign(formData, detail);
    }
});

async function handleSubmit() {
    try {
        await formRef.value?.validate();
        await api.save(formData as any);
        uni.showToast({ title: isEdit.value ? '更新成功' : '创建成功' });
        setTimeout(() => uni.navigateBack(), 1500);
    } catch (e) {
        // 校验失败
    }
}

function handleCancel() {
    uni.navigateBack();
}

```

```
// @jnpf-gen:insert-point=custom-logic
// @jnpf-gen:end-insert-point=custom-logic
</script>

<style scoped lang="scss">
.page-form {
  padding: 24rpx;
}
.form-actions {
  margin-top: 48rpx;
}
</style>
```

Week 2: UniApp 编译器核心

F-7.4 UniApp 编译器（标准 uni-app）

```
/**
 * UniApp 代码生成器（标准 uni-app，目标：小程序端）
 *
 * 输入：FormPageIR（与 Vue3 编译器使用同一个 IR）
 * 输出：完整的 uni-app 项目文件集合
 *
 * 生成产物：
 *   pages/{entity}/list.vue      - 列表页
 *   pages/{entity}/form.vue     - 表单页
 *   pages/{entity}/detail.vue   - 详情页
 *   api/{entity}.ts             - Alova API
 *   stores/{entity}.ts          - Pinia Store
 *   types/{entity}.ts           - TypeScript 类型
 *   pages.json                  - 路由配置（追加）
 *   manifest.json               - 应用配置（追加）
 */

import type { FormPageIR } from '../..ir/types';
import type { CompilerConfig, GeneratedProject, CompileResult } from '../vue3/types';
import { registry } from '../..component-registry';

// EJS 模板引擎（用于渲染页面模板）
import ejs from 'ejs';

export class UniAppCompiler {
  private config: CompilerConfig;
  private platform: 'mp-weixin' | 'mp-alipay' | 'mp-douyin' | 'h5';

  constructor(config: Partial<CompilerConfig> & { entity: string }, platform: 'mp-weixin' | 'mp-alipay' | 'mp-douyin' | 'h5' = 'mp-weixin') {
    this.config = {
      entity: config.entity,
      entityLabel: config.entityLabel ?? config.entity,
      apiBasePath: config.apiBasePath ?? `/api/${config.entity}`,
      generatorVersion: config.generatorVersion ?? '1.0.0',
    };
    this.platform = platform;
  }

  compile(ir: FormPageIR): CompileResult {
    const project: GeneratedProject = new Map();
```



```

const warnings: string[] = [];
const complexExpressions: string[] = [];
const e = this.config.entity;
const E = this.capitalize(e);
const now = new Date().toISOString();

// 检查复杂表达式
for (const expr of ir.expressions) {
  if (expr.level === 'complex') {
    complexExpressions.push(`${expr.id}: ${expr.body.slice(0, 100)}`);
    warnings.push(`表达式 ${expr.id} 为复杂级别，需人工迁移`);
  }
}

// 1. types
project.set(`types/${e}.ts`, this.generateTypes(ir));

// 2. api (Alova)
project.set(`api/${e}.ts`, this.generateApi(ir));

// 3. store (Pinia)
project.set(`stores/${e}.ts`, this.generateStore(ir));

// 4. 列表页
project.set(`pages/${e}/list.vue`, this.generateListPage(ir));

// 5. 表单页
project.set(`pages/${e}/form.vue`, this.generateFormPage(ir));

// 6. 详情页
project.set(`pages/${e}/detail.vue`, this.generateDetailPage(ir));

// 7. pages.json 片段 (追加到已有配置)
project.set(`pages-${e}.json`, this.generatePagesJson(ir));

return { project, warnings, complexExpressions };
}

private generateTypes(ir: FormPageIR): string {
  // 复用 vue3 编译器的类型生成逻辑
  // 从 ../vue3/type-gen.ts 导入
  const entity = this.capitalize(this.config.entity);
  const lines: string[] = [];
  lines.push(`// @jnpf-generated v${this.config.generatorVersion}`);
  lines.push(`entity=${this.config.entity} platform=${this.platform}`);
  lines.push(`/* eslint-disable */`);
  lines.push('');
  lines.push(`export interface ${entity}Entity {`);
  for (const field of ir.fields) {
    const tsType = this.mapFieldToTsType(field);
    const optional = field.config.required ? '' : '?';
    lines.push(`  /** ${field.label} */`);
    lines.push(`  ${field.model}${optional}: ${tsType};`);
  }
  lines.push('}');
  return lines.join('\n');
}

private generateApi(ir: FormPageIR): string {
  const entity = this.capitalize(this.config.entity);

```

```

    return `// @jnpf-generated v${this.config.generatorVersion}
entity=${this.config.entity} platform=${this.platform}
/* eslint-disable */
import { createEntityApi } from '@api/request';
import type { ${entity}Entity } from '@types/${this.config.entity}';

const api = createEntityApi<${entity}Entity>(`${this.config.apiBasePath}`);

export default api;

export const {
  list: get${entity}List,
  detail: get${entity}Detail,
  create: create${entity},
  update: update${entity},
  delete: delete${entity},
  batchDelete: batchDelete${entity},
} = api;
`;
}

private generateStore(ir: FormPageIR): string {
  const entity = this.capitalize(this.config.entity);
  return `// @jnpf-generated v${this.config.generatorVersion}
entity=${this.config.entity} platform=${this.platform}
/* eslint-disable */
import { defineStore } from 'pinia';
import { ref, reactive } from 'vue';
import api from '@api/${this.config.entity}';

export const use${entity}Store = defineStore(`${this.config.entity}`, () => {
  const loading = ref(false);
  const list = ref<${entity}Entity[]>([]);
  const current = ref<${entity}Entity | undefined>();
  const pagination = reactive({ current: 1, pageSize: 20, total: 0 });

  async function loadList(params?: Record<string, unknown>) {
    loading.value = true;
    try {
      list.value = await api.list({ currentPage: pagination.current, pageSize:
pagination.pageSize, ...params });
    } finally {
      loading.value = false;
    }
  }

  async function loadDetail(id: string) {
    loading.value = true;
    try {
      current.value = await api.detail(id);
    } finally {
      loading.value = false;
    }
  }

  async function save(data: Partial<${entity}Entity>) {
    if (current.value?.id) {
      await api.update(current.value.id, data);
    } else {
      await api.create(data);
    }
  }
}
`

```

```

    }
  }

  async function remove(id: string) {
    await api.delete(id);
    await loadList();
  }

  return { loading, list, current, pagination, loadList, loadDetail, save, remove };
});
`;
}

private generateListPage(ir: FormPageIR): string {
  // 使用 EJS 模板渲染列表页
  // 模板内容在 F-7.3 中已给出
  // 工程师将 F-7.3 的模板存为 templates/uniapp/list.vue.ejs
  // 这里用 ejs.render() 渲染

  const entity = this.capitalize(this.config.entity);
  const searchFields = ir.listConfig?.searchFields ?? [];
  const displayField = ir.fields[0]?.model || 'id';
  const now = new Date().toISOString();

  // 简化版（工程师后续用 EJS 模板替换）
  return `<!-- @jnpf-generated v${this.config.generatorVersion}
entity=${this.config.entity} platform=${this.platform} -->
<!-- 生成时间: ${now} -->
<template>
  <view class="page-list">
    <view class="search-bar">
      ${searchFields.map(sf => `      <wd-input v-model="searchParams.${sf.field}"
placeholder="请输入${sf.label}" clearable />`).join('\n')}`
      <view class="search-actions">
        <wd-button type="primary" size="small" @click="handleSearch">查询</wd-button>
        <wd-button size="small" @click="handleReset">重置</wd-button>
      </view>
    </view>
    <wd-cell-group>
      <wd-cell>
        v-for="item in store.list"
        :key="item.id"
        :title="String(item.${displayField} ?? '')"
        @click="handleDetail(item)"
      >
        <template #value>
          <view class="cell-actions">
            <wd-button size="mini" @click.stop="handleEdit(item)">编辑</wd-button>
            <wd-button size="mini" type="error" @click.stop="handleDelete(item)">删除
</wd-button>
          </view>
        </template>
      </wd-cell>
    </wd-cell-group>
    <wd-status-tip v-if="!store.loading && store.list.length === 0" tip="暂无数据" />
    <view class="fab" @click="handleAdd"><wd-icon name="add" size="24px" /></view>
  </view>
</template>

<script setup lang="ts">

```

```

import { onMounted, reactive } from 'vue';
import { use${entity}Store } from '@/stores/${this.config.entity}';

const store = use${entity}Store();
const searchParams = reactive<Record<string, string>>({
  ${searchFields.map(sf => `  ${sf.field}: '',`).join('\n')}
});

onMounted(() => store.loadList());

function handleSearch() {
  Object.assign(store.searchParams, searchParams);
  store.pagination.current = 1;
  store.loadList();
}
function handleReset() {
  Object.keys(searchParams).forEach(k => searchParams[k] = '');
  handleSearch();
}
function handleAdd() { uni.navigateTo({ url: '/pages/${this.config.entity}/form' }); }
function handleEdit(item) { uni.navigateTo({ url: '/pages/${this.config.entity}/form?id=' + item.id }); }
function handleDetail(item) { uni.navigateTo({ url: '/pages/${this.config.entity}/detail?id=' + item.id }); }
async function handleDelete(item) {
  uni.showModal({
    title: '确认删除',
    content: '确定删除?',
    success: async (res) => {
      if (res.confirm) { await store.remove(item.id); uni.showToast({ title: '删除成功'
}); }
    },
  });
}
</script>
`;
}

```

```

private generateFormPage(ir: FormPageIR): string {
  const entity = this.capitalize(this.config.entity);
  const now = new Date().toISOString();

  return `<!-- @jnpf-generated v${this.config.generatorVersion}
entity=${this.config.entity} platform=${this.platform} -->
<!-- 生成时间: ${now} -->
<template>
  <view class="page-form">
    <wd-form ref="formRef" :model="formData">
${ir.fields.map(f => {
  const app = f.component.app;
  let widget = '';
  if (app === 'uni-data-select') {
    widget = `      <wd-cell title="${f.label}"${f.config.required ? ' required' : ''}>
<wd-select-picker v-model="formData.${f.model}" placeholder="请选择${f.label}" /></wd-
cell>`;
  } else if (app === 'uni-datetime-picker') {
    widget = `      <wd-cell title="${f.label}"${f.config.required ? ' required' : ''}>
<wd-datetime-picker v-model="formData.${f.model}" placeholder="请选择${f.label}" /></wd-
cell>`;
  } else if (app === 'switch') {

```

```

        widget = `      <wd-cell title="{{$f.label}}"><wd-switch v-model="formData.{{$f.model}}"
/></wd-cell>`;
    } else {
        widget = `      <wd-cell title="{{$f.label}}"{{$f.config.required ? ' required' : ''}}>
<wd-input v-model="formData.{{$f.model}}" placeholder="请输入{{$f.label}}" /></wd-cell>`;
    }
    return widget;
}).join('\n')}
</wd-form>
<!-- @jnpf-gen:insert-point=custom-form-fields -->
<!-- @jnpf-gen:end-insert-point=custom-form-fields -->
<view class="form-actions">
    <wd-button type="primary" block @click="handleSubmit">提交</wd-button>
    <wd-button block @click="handleCancel" style="margin-top: 16rpx">取消</wd-button>
</view>
</view>
</template>

<script setup lang="ts">
import { ref, reactive, onMounted } from 'vue';
import api from '@api/{{$this.config.entity}}';

const formRef = ref();
const isEdit = ref(false);
const formData = reactive<Record<string, unknown>>(<{
  {{$ir.fields.map(f => `  {{$f.model}}: {{$this.getDefaultvalue(f)},`}).join('\n')}}
});

onMounted(async () => {
    const pages = getCurrentPages();
    const page = pages[pages.length - 1];
    const id = page?.options?.id;
    if (id) {
        isEdit.value = true;
        const detail = await api.detail(id);
        Object.assign(formData, detail);
    }
});

async function handleSubmit() {
    try {
        await formRef.value?.validate();
        await api.save(formData);
        uni.showToast({ title: isEdit.value ? '更新成功' : '创建成功' });
        setTimeout(() => uni.navigateBack(), 1500);
    } catch (e) { /* 校验失败 */ }
}

function handleCancel() { uni.navigateBack(); }

// @jnpf-gen:insert-point=custom-logic
// @jnpf-gen:end-insert-point=custom-logic
</script>
`;
}

private generateDetailPage(ir: FormPageIR): string {
    const entity = this.capitalize(this.config.entity);
    return `<!-- @jnpf-generated v${this.config.generatorVersion}
entity=${this.config.entity} platform=${this.platform} -->

```

```

<template>
  <view class="page-detail">
    <wd-cell-group title="{{this.config.entityLabel}}详情">
      {{ir.fields.map(f => `      <wd-cell title="{{f.label}}" :value="String(data.${f.model}
      ?? '')" />`).join('\n')}}
    </wd-cell-group>
    <!-- @jnpf-gen:insert-point=custom-detail-fields -->
    <!-- @jnpf-gen:end-insert-point=custom-detail-fields -->
  </view>
</template>

<script setup lang="ts">
import { ref, onMounted } from 'vue';
import api from '@api/{{this.config.entity}}';

const data = ref<Record<string, unknown>>({});

onMounted(async () => {
  const pages = getCurrentPages();
  const page = pages[pages.length - 1];
  const id = page?.options?.id;
  if (id) data.value = await api.detail(id);
});
</script>
`;
}

private generatePagesJson(ir: FormPageIR): string {
  const e = this.config.entity;
  return JSON.stringify({
    pages: [
      { path: `pages/${e}/list`, style: { navigationBarTitleText:
`${this.config.entityLabel}}列表` } },
      { path: `pages/${e}/form`, style: { navigationBarTitleText:
`${this.config.entityLabel}}表单` } },
      { path: `pages/${e}/detail`, style: { navigationBarTitleText:
`${this.config.entityLabel}}详情` } },
    ],
    null, 2);
}

// ---- 工具方法 ----

private capitalize(s: string): string {
  return s.charAt(0).toUpperCase() + s.slice(1);
}

private mapFieldToTSType(field: any): string {
  const typeMap: Record<string, string> = {
    'JnpfInput': 'string', 'JnpfTextarea': 'string', 'JnpfInputNumber': 'number',
    'JnpfSwitch': 'boolean', 'JnpfDatePicker': 'string', 'JnpfTimePicker': 'string',
    'JnpfRate': 'number', 'JnpfSlider': 'number',
    'JnpfSelect': field.config.multiple ? 'string[]' : 'string',
    'JnpfRadio': 'string', 'JnpfCheckbox': 'string[]',
  };
  return typeMap[field.component.jnpfKey] || 'unknown';
}

private getDefaultValue(field: any): string {

```

```

    if (field.config.defaultValue !== null) return
JSON.stringify(field.config.defaultValue);
    const typeMap: Record<string, string> = {
        'JnpfInput': '""', 'JnpfTextarea': '""', 'JnpfInputNumber': '0',
        'JnpfSwitch': 'false', 'JnpfDatePicker': '""', 'JnpfSelect': field.config.multiple
? '[]' : '""',
        'JnpfRadio': '""', 'JnpfCheckbox': '[]',
    };
    return typeMap[field.component.jnpfkey] || "";
}
}

```

Week 3: FlowIR v1 + pages.json 生成器 (F-7.9 并行)

F-7.5 UniApp X: 暂缓 (v5.0 创始人裁定)。本节接口规格保留于仓库设计文档，**不纳入阶段三验收**；Week 3 施工重心转为 **FlowIR v1** (见「专家审阅裁定·P0 新增施工包」)。

F-7.5 UniApp X 编译器 (App 端 · 暂缓，接口保留)

```

/**
 * UniApp X 代码生成器 (uvue + uts, 目标: App 端)
 *
 * 与 UniAppCompiler 的核心区别:
 * 1. 文件扩展名: .vue → .uvue
 * 2. 脚本语言: TypeScript → UTS (编译为 Kotlin/Swift)
 * 3. 组件库: wot-design-uni → uvue 原生组件
 * 4. 平台 API: uni.xxx → 可直接调用原生 API
 *
 * 复用: API 层、Store 层、类型层 与标准 UniApp 完全相同
 */

import type { FormPageIR } from '../..//ir/types';
import type { CompilerConfig, GeneratedProject, CompileResult } from '../vue3/types';

export class UniAppXCompiler {
    private config: CompilerConfig;

    constructor(config: Partial<CompilerConfig> & { entity: string }) {
        this.config = {
            entity: config.entity,
            entityLabel: config.entityLabel ?? config.entity,
            apiBasePath: config.apiBasePath ?? `/api/${config.entity}`,
            generatorVersion: config.generatorVersion ?? '1.0.0',
        };
    }

    compile(ir: FormPageIR): CompileResult {
        const project: GeneratedProject = new Map();
        const warnings: string[] = [];
        const complexExpressions: string[] = [];
        const e = this.config.entity;
        const now = new Date().toISOString();

        // 共用层 (与标准 UniApp 完全相同)
        project.set(`types/${e}.ts`, this.generateTypes(ir));
        project.set(`api/${e}.ts`, this.generateApi(ir));
        project.set(`stores/${e}.ts`, this.generateStore(ir));
    }
}

```

```

// 差异层 (.uvue 文件)
project.set(`pages/${e}/list.uvue`, this.generateListPage(ir));
project.set(`pages/${e}/form.uvue`, this.generateFormPage(ir));
project.set(`pages/${e}/detail.uvue`, this.generateDetailPage(ir));

// pages.json
project.set(`pages-${e}.json`, this.generatePagesJson(ir));

return { project, warnings, complexExpressions };
}

private generateListPage(ir: FormPageIR): string {
  const entity = this.capitalize(this.config.entity);
  const searchFields = ir.listConfig?.searchFields ?? [];
  const displayField = ir.fields[0]?.model || 'id';
  const now = new Date().toISOString();

  // .uvue 文件使用 uvue 原生组件
  // 与 .vue 的区别:
  // 1. 使用 <view> 而非 HTML 标签
  // 2. 组件来自 uvue 组件库而非 wot-design
  // 3. 样式使用 rpx 单位
  return `<!-- @jnpf-generated v${this.config.generatorVersion}
entity=${this.config.entity} platform=app-x -->
<!-- 生成时间: ${now} -->

<template>
  <view class="page-list">
    <view class="search-bar">
      ${searchFields.map(sf => `      <input class="search-input" v-
model="searchParams.${sf.field}" placeholder="请输入 ${sf.label}" />`).join('\n')}`
    <view class="search-actions">
      <button class="btn-primary" @click="handleSearch">查询</button>
      <button @click="handleReset">重置</button>
    </view>
  </view>

  <list class="data-list">
    <cell v-for="(item, index) in list" :key="item.id ?? index"
@click="handleDetail(item)">
      <view class="list-item">
        <text class="item-title">{{ item.${displayField} }}</text>
        <view class="item-actions">
          <text class="btn-edit" @click.stop="handleEdit(item)">编辑</text>
          <text class="btn-delete" @click.stop="handleDelete(item)">删除</text>
        </view>
      </view>
    </cell>
  </list>

  <view v-if="!loading && list.length === 0" class="empty">
    <text>暂无数据</text>
  </view>

  <view class="fab" @click="handleAdd">
    <text class="fab-icon">+</text>
  </view>
</view>
</template>

```



```

<script lang="uts">
// UTS 脚本（编译为 Kotlin/Swift）
import { ref, reactive, onMounted } from 'vue'
import api from '@api/${this.config.entity}'

export default {
  setup() {
    const loading = ref(false)
    const list = ref<${entity}Entity[]>([])
    const searchParams = reactive<Record<string, string>>({
      ${searchFields.map(sf => `      ${sf.field}: '',`).join('\n')}
    })

    onMounted(() => { loadList() })

    async function loadList() {
      loading.value = true
      try {
        list.value = await api.list({ currentPage: 1, pageSize: 20, ...searchParams })
      } finally {
        loading.value = false
      }
    }

    function handleSearch() { loadList() }
    function handleReset() {
      Object.keys(searchParams).forEach(k => { searchParams[k] = '' })
      handleSearch()
    }
    function handleAdd() { uni.navigateTo({ url: '/pages/${this.config.entity}/form' }) }

    function handleEdit(item : any) { uni.navigateTo({ url:
'/pages/${this.config.entity}/form?id=' + item.id }) }
    function handleDetail(item : any) { uni.navigateTo({ url:
'/pages/${this.config.entity}/detail?id=' + item.id }) }
    async function handleDelete(item : any) {
      const res = await uni.showModal({ title: '确认删除', content: '确定删除? ' })
      if (res.confirm) {
        await api.delete(item.id)
        uni.showToast({ title: '删除成功' })
        loadList()
      }
    }

    return { loading, list, searchParams, handleSearch, handleReset, handleAdd,
handleEdit, handleDetail, handleDelete }
  }
}
</script>

<style>
.page-list { padding: 20px; }
.search-bar { margin-bottom: 20px; }
.search-input { height: 40px; border: 1px solid #ddd; border-radius: 8px; padding: 0
12px; margin-bottom: 8px; }
.search-actions { display: flex; flex-direction: row; gap: 12px; }
.btn-primary { background-color: #0083ff; color: #ffffff; border-radius: 8px; padding:
8px 16px; }
.list-item { display: flex; flex-direction: row; justify-content: space-between; align-
items: center; padding: 16px; border-bottom: 1px solid #f0f0f0; }

```

```

.item-title { font-size: 16px; color: #333; }
.item-actions { display: flex; flex-direction: row; gap: 16px; }
.btn-edit { color: #0083ff; font-size: 14px; }
.btn-delete { color: #ff4d4f; font-size: 14px; }
.empty { display: flex; justify-content: center; align-items: center; padding: 80px 0; color: #999; }
.fab { position: fixed; right: 30px; bottom: 100px; width: 50px; height: 50px; border-radius: 25px; background-color: #0083ff; display: flex; justify-content: center; align-items: center; }
.fab-icon { color: #ffffff; font-size: 24px; }
</style>
`;
}

private generateFormPage(ir: FormPageIR): string {
    // 类似标准 UniApp 的表单页，但使用原生组件
    // 工程师按 generateListPage 的模式实现
    return `<!-- @jnpf-generated form.uvue for ${this.config.entity} - 待工程师实现 -->`;
}

private generateDetailPage(ir: FormPageIR): string {
    return `<!-- @jnpf-generated detail.uvue for ${this.config.entity} - 待工程师实现 -->`;
}

// 以下方法与 UniAppCompiler 相同（共用层）
private generateTypes(ir: FormPageIR): string { /* 同 UniAppCompiler */ return ''; }
private generateApi(ir: FormPageIR): string { /* 同 UniAppCompiler */ return ''; }
private generateStore(ir: FormPageIR): string { /* 同 UniAppCompiler */ return ''; }
private generatePagesJson(ir: FormPageIR): string { /* 同 UniAppCompiler */ return ''; }
}
private capitalize(s: string): string { return s.charAt(0).toUpperCase() + s.slice(1); }
}
}

```

F-7.6 pages.json 智能合并器

```

/**
 * pages.json 智能合并器
 *
 * 问题：多个实体的页面需要合并到同一个 pages.json 中
 * 方案：编译器为每个实体生成 pages-{entity}.json 片段
 *       合并器将所有片段合并为完整的 pages.json
 */

export function mergePagesJson(fragments: string[]): string {
    const allPages: unknown[] = [];
    const allSubPackages: unknown[] = [];

    for (const fragment of fragments) {
        try {
            const parsed = JSON.parse(fragment);
            if (parsed.pages) allPages.push(...parsed.pages);
            if (parsed.subPackages) allSubPackages.push(...parsed.subPackages);
        } catch (e) {
            console.warn('[pages-json-merger] 解析失败:', e);
        }
    }

    return JSON.stringify({

```

```

pages: allPages,
subPackages: allSubPackages.length > 0 ? allSubPackages : undefined,
globalStyle: {
  navigationStyle: 'default',
  navigationBarTitleText: 'JNPF',
  navigationBarBackgroundColor: '#0083ff',
  navigationBarTextStyle: 'white',
},
tabBar: undefined, // 由应用层配置
}, null, 2);
}

```

F-7.7 测试

```
// src/core/compiler/__tests__/uniapp-compiler.test.ts
```

```

import { describe, it, expect } from 'vitest';
import { UniAppCompiler } from '../uniapp/compiler';
import { UniAppXCompiler } from '../uniapp-x/compiler';
import { cleanSchema } from '../../ir/schema-cleaner';
import { mergePagesJson } from '../uniapp/pages-json-merger';

const minimalSchema = {
  data: {
    formData: JSON.stringify({
      fields: [
        { __vModel__: 'name', __config__: { label: '姓名', tag: 'JnpfInput', jnpfKey:
'JnpfInput', required: true } },
        { __vModel__: 'age', __config__: { label: '年龄', tag: 'JnpfInputNumber',
jnpfKey: 'JnpfInputNumber' } },
      ],
      funcs: {},
      virtualFieldList: [],
    }),
  },
};

describe('UniAppCompiler (小程序端)', () => {
  const ir = cleanSchema(minimalSchema);
  const compiler = new UniAppCompiler({ entity: 'student', entityLabel: '学生' }, 'mp-
weixin');
  const result = compiler.compile(ir);

  it('生成列表页 (.vue)', () => {
    expect(result.project.has('pages/student/list.vue')).toBe(true);
    const list = result.project.get('pages/student/list.vue')!;
    expect(list).toContain('wd-cell');
    expect(list).toContain('@jnpf-generated');
    expect(list).toContain('platform=mp-weixin');
  });

  it('生成表单页 (.vue)', () => {
    expect(result.project.has('pages/student/form.vue')).toBe(true);
    const form = result.project.get('pages/student/form.vue')!;
    expect(form).toContain('wd-form');
    expect(form).toContain('@jnpf-gen:insert-point');
  });

  it('生成 API (Alova)', () => {
    expect(result.project.has('api/student.ts')).toBe(true);
  });
}

```

```
});

it('生成 Store (Pinia)', () => {
  expect(result.project.has('stores/student.ts')).toBe(true);
});

it('生成 types', () => {
  expect(result.project.has('types/student.ts')).toBe(true);
});

it('生成 pages.json 片段', () => {
  expect(result.project.has('pages-student.json')).toBe(true);
  const pagesJson = JSON.parse(result.project.get('pages-student.json')!);
  expect(pagesJson.pages.length).toBe(3);
});

it('零 eval/Function', () => {
  for (const [, content] of result.project) {
    expect(content).not.toMatch(/\beval\b/);
    expect(content).not.toMatch(/new Function/);
  }
});

describe('UniAppXCompiler (App 端)', () => {
  const ir = cleanSchema(minimalSchema);
  const compiler = new UniAppXCompiler({ entity: 'student', entityLabel: '学生' });
  const result = compiler.compile(ir);

  it('生成列表页 (.uvue)', () => {
    expect(result.project.has('pages/student/list.uvue')).toBe(true);
    const list = result.project.get('pages/student/list.uvue')!;
    expect(list).toContain('<list>');
    expect(list).toContain('<cell>');
    expect(list).toContain('platform=app-x');
  });

  it('共用层与标准 UniApp 相同', () => {
    expect(result.project.has('api/student.ts')).toBe(true);
    expect(result.project.has('stores/student.ts')).toBe(true);
    expect(result.project.has('types/student.ts')).toBe(true);
  });
});

describe('pages.json 合并器', () => {
  it('合并多个实体的 pages.json 片段', () => {
    const fragment1 = JSON.stringify({ pages: [{ path: 'pages/student/list' }] });
    const fragment2 = JSON.stringify({ pages: [{ path: 'pages/order/list' }] });
    const merged = mergePagesJson([fragment1, fragment2]);
    const parsed = JSON.parse(merged);
    expect(parsed.pages.length).toBe(2);
  });
});
```

Week 4-5: 后端清零收尾

后端清零收尾任务：

App.GetService 最后 7 处：

Sprint 6: 第三方集成模块 (4 处)

Sprint 7: 剩余零散调用 (3 处)

累计: 37/37 → 清零 

CreateScope 最后 14 处：

Sprint 6-7: 按模块逐一清理

累计: 24/24 → 清零 

#pragma warning disable 清零：

随 App.GetService 和 CreateScope 消化同步移除

累计: 68/68 → 清零 

Analyzer 存量看板更新：

App.GetService: 37 → 0 

CreateScope: 24 → 0 

#pragma warning disable: 68 → 0 

async void (非豁免): 0  (已清零)

JWT 权限白名单收紧：

Sprint 6: 为核心业务模块强制校验

Sprint 7: 白名单仅保留公开端点

测试覆盖率：

目标: 框架层 60%, 核心业务 40%

Sprint 6-7: 为修改的代码补充单元测试

阶段三交付物

F-7 UniApp 代码生成器：

- src/core/compiler/uniapp/compiler.ts – 标准 UniApp 编译器
- src/core/compiler/uniapp-x/compiler.ts – UniApp X 编译器
- src/core/compiler/uniapp/pages-json-merger.ts – pages.json 合并器
- src/api/request.ts – Alova 通用请求封装
- src/core/compiler/__tests__/uniapp-compiler.test.ts – 测试
- 标签: v5.2-uniapp-m1

后端清零收尾：

- App.GetService: 37 → 0
- CreateScope: 24 → 0
- #pragma warning disable: 68 → 0
- Analyzer 存量看板更新
- JWT 权限白名单收紧完成
- 标签: v5.2-backend-cleanup-complete

阶段三里程碑验收

- UniApp 编译器通过测试（标准 + X 两个模式）
- 生成的列表页/表单页/详情页包含正确的组件和数据绑定
- pages.json 合并器正确合并多个实体
- 共用层（API + Store + types）两种模式完全一致
- 零 eval/Function
- 后端 App.GetService 37 → 0
- 后端 CreateScope 24 → 0
- 后端 #pragma warning disable 68 → 0
- Analyzer 存量看板全部归零
- JWT 权限白名单收紧完成

阶段三完成。说“继续”我贴出阶段四（平台整合 + 下载源码 + 回写 + 多目标编译器统一入口）。

阶段四：平台整合 + 下载源码 + 回写 + 多目标编译器统一入口（3 周）

目标

将前三个阶段的所有编译器统一为一个平台级入口，实现“一键生成、多端输出”的能力，同时实现“下载源码”和“回写平台”的双向通道。

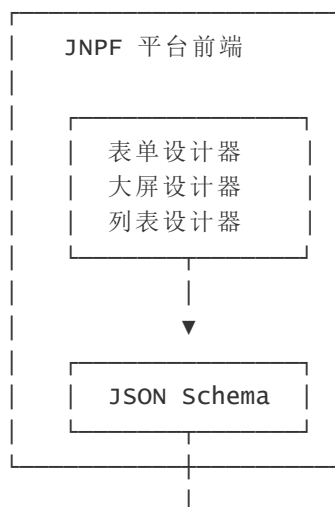
这是手工低代码平台的“顶峰一跃”——用户在 JNPF 平台上配置好一切，一键生成：

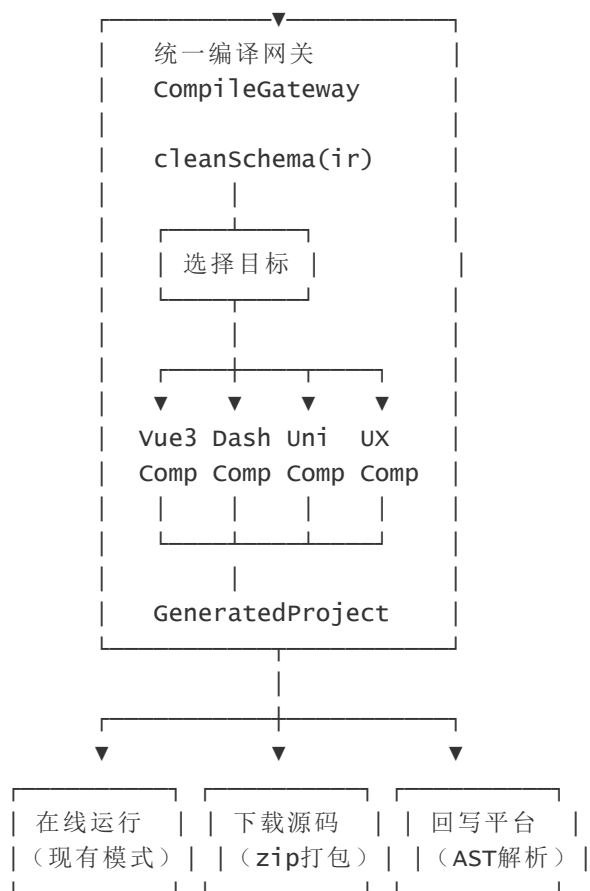
- ① Vue3 web 项目（可独立运行）
- ② 数字大屏项目（含 3D 数字孪生）
- ③ UniApp 小程序项目
- ④ UniApp X App 项目

用户可以选择：

- A. 在线运行（现有模式，不改变）
- B. 下载源码（新能力，在 IDE 中二次开发）
- C. 回写平台（新能力，IDE 修改后同步回平台）

统一架构





Week 1: 统一编译网关

F-8.1 编译目标枚举

```
/**
 * 编译目标定义
 *
 * 每个目标对应一个编译器实例
 * 所有编译器共享同一个 IR 输入
 */

import type { FormPageIR, DashboardIR } from '../..ir/types';
import type { DashboardIR } from '../..ir/dashboard-types';
import type { CompileResult } from '../vue3/types';

/** 编译目标 */
export type CompileTarget =
  | 'vue3-web' // Vue3 web 应用（表单 + 列表 CRUD）
  | 'dashboard' // 数字大屏（图表 + 装饰 + 布局）
  | 'dashboard-3d' // 数字大屏 + 3D 数字孪生（VIP）
  | 'uniapp-weixin' // UniApp 微信小程序
  | 'uniapp-alipay' // UniApp 支付宝小程序
  | 'uniapp-douyin' // UniApp 抖音小程序
  | 'uniapp-h5' // UniApp H5
  | 'uniapp-x-app'; // UniApp X 原生 App

/** 编译目标元数据 */
export interface CompileTargetMeta {
  id: CompileTarget;
  name: string;
  description: string;
```

```

icon: string;
/** 是否为 VIP 功能 */
vip: boolean;
/** 输入 IR 类型 */
irType: 'form' | 'dashboard';
/** 输出文件扩展名列表 */
outputExtensions: string[];
}

/** 所有可用编译目标 */
export const COMPILER_TARGETS: Record<CompileTarget, CompileTargetMeta> = {
  'vue3-web': {
    id: 'vue3-web',
    name: 'Vue3 web 应用',
    description: '标准 Vue3 + Ant Design Vue web 应用, 可独立运行',
    icon: 'vue',
    vip: false,
    irType: 'form',
    outputExtensions: ['.vue', '.ts'],
  },
  'dashboard': {
    id: 'dashboard',
    name: '数字大屏',
    description: 'Vue3 + ECharts 数据大屏, 支持图表、装饰、实时数据',
    icon: 'dashboard',
    vip: false,
    irType: 'dashboard',
    outputExtensions: ['.vue', '.ts', '.css', '.json'],
  },
  'dashboard-3d': {
    id: 'dashboard-3d',
    name: '3D 数字孪生大屏',
    description: '含 Three.js 3D 场景、POI、飞线、围栏、热力图',
    icon: '3d',
    vip: true,
    irType: 'dashboard',
    outputExtensions: ['.vue', '.ts', '.css', '.json'],
  },
  'uniapp-weixin': {
    id: 'uniapp-weixin',
    name: '微信小程序',
    description: '标准 uni-app 微信小程序, wot-design-uni 组件库',
    icon: 'wechat',
    vip: false,
    irType: 'form',
    outputExtensions: ['.vue', '.ts', '.json'],
  },
  'uniapp-alipay': {
    id: 'uniapp-alipay',
    name: '支付宝小程序',
    description: '标准 uni-app 支付宝小程序',
    icon: 'alipay',
    vip: false,
    irType: 'form',
    outputExtensions: ['.vue', '.ts', '.json'],
  },
  'uniapp-douyin': {
    id: 'uniapp-douyin',
    name: '抖音小程序',
    description: '标准 uni-app 抖音小程序',
  }
}

```



```

    icon: 'douyin',
    vip: false,
    irType: 'form',
    outputExtensions: ['.vue', '.ts', '.json'],
  },
  'uniapp-h5': {
    id: 'uniapp-h5',
    name: 'H5 移动端',
    description: '标准 uni-app H5, 适配移动浏览器',
    icon: 'h5',
    vip: false,
    irType: 'form',
    outputExtensions: ['.vue', '.ts', '.json'],
  },
  'uniapp-x-app': {
    id: 'uniapp-x-app',
    name: '原生 App',
    description: 'uni-app X (uvue + uts), 编译为 Kotlin/Swift 原生应用',
    icon: 'app',
    vip: true,
    irType: 'form',
    outputExtensions: ['.uvue', '.uts', '.ts', '.json'],
  },
};

```

F-8.2 统一编译网关

```

/**
 * 统一编译网关
 *
 * 所有编译器的统一入口
 * 负责:
 *   1. 接收 JSON Schema
 *   2. 清洗为 IR
 *   3. 根据目标选择编译器
 *   4. 执行编译
 *   5. 返回 GeneratedProject
 */

import type { FormPageIR } from '../..ir/types';
import type { DashboardIR } from '../..ir/dashboard-types';
import type { CompileTarget } from './targets';
import type { CompilerConfig, CompileResult, GeneratedProject } from './vue3/types';

import { cleanSchema } from '../..ir/schema-cleaner';
import { validateIR, hasErrors } from '../..ir/validator';
import { Vue3Compiler } from './vue3/compiler';
import { DashboardCompiler } from './dashboard/compiler';
import { UniAppCompiler } from './uniapp/compiler';
import { UniAppXCompiler } from './uniapp-x/compiler';
import { COMPILE_TARGETS } from './targets';

export interface CompileRequest {
  /** 原始 JSON Schema (来自 JNPF 平台) */
  schema: unknown;
  /** 编译目标 */
  target: CompileTarget;
  /** 编译配置 */
  config: Partial<CompilerConfig> & { entity: string };
  /** 是否包含 3D (仅 dashboard 目标有效) */

```

```

    include3D?: boolean;
}

export interface CompileResponse {
    /** 是否成功 */
    success: boolean;
    /** 生成的项目文件 */
    project?: GeneratedProject;
    /** IR 验证问题 */
    issues?: { level: string; path: string; message: string }[];
    /** 编译警告 */
    warnings?: string[];
    /** 复杂表达式列表（需人工迁移） */
    complexExpressions?: string[];
    /** 错误信息 */
    error?: string;
}

/**
 * 统一编译网关
 */
export async function compileGateway(request: CompileRequest): Promise<CompileResponse>
{
    try {
        // Step 1: 清洗 Schema → IR
        const ir = cleanSchema(request.schema);

        // Step 2: 验证 IR
        const issues = validateIR(ir);
        const errors = issues.filter(i => i.level === 'error');
        if (errors.length > 0) {
            return {
                success: false,
                issues,
                error: `IR 验证失败: ${errors.length} 个错误`,
            };
        }

        // Step 3: 根据目标选择编译器
        const targetMeta = COMPILE_TARGETS[request.target];
        if (!targetMeta) {
            return { success: false, error: `未知编译目标: ${request.target}` };
        }

        let result: CompileResult;

        switch (request.target) {
            case 'vue3-web': {
                const compiler = new Vue3Compiler(request.config);
                result = compiler.compile(ir);
                break;
            }

            case 'dashboard':
            case 'dashboard-3d': {
                if (ir.type !== 'form') {
                    // 如果输入是 DashboardIR 直接使用
                    const dashIR = ir as unknown as DashboardIR;
                    const compiler = new DashboardCompiler(request.config);
                    result = compiler.compile(dashIR);
                }
            }
        }
    }
}

```

```

    } else {
      // 表单 IR 转换为大屏 IR (简单场景)
      return { success: false, error: '大屏编译需要 DashboardIR, 当前输入为
FormPageIR' };
    }
    break;
  }

  case 'uniapp-weixin':
  case 'uniapp-alipay':
  case 'uniapp-douyin':
  case 'uniapp-h5': {
    const platform = request.target.replace('uniapp-', '') as 'mp-weixin' | 'mp-
alipay' | 'mp-douyin' | 'h5';
    const compiler = new UniAppCompiler(request.config, platform);
    result = compiler.compile(ir);
    break;
  }

  case 'uniapp-x-app': {
    const compiler = new UniAppXCompiler(request.config);
    result = compiler.compile(ir);
    break;
  }

  default:
    return { success: false, error: `编译目标 ${request.target} 尚未实现` };
}

return {
  success: true,
  project: result.project,
  issues,
  warnings: result.warnings,
  complexExpressions: result.complexExpressions,
};

} catch (e) {
  return {
    success: false,
    error: `编译失败: ${(e as Error).message}`,
  };
}
}

/**
 * 批量编译 (同时生成多个目标)
 */
export async function compileMultiTarget(
  schema: unknown,
  targets: CompileTarget[],
  config: Partial<CompilerConfig> & { entity: string }
): Promise<Map<CompileTarget, CompileResponse>> {
  const results = new Map<CompileTarget, CompileResponse>();

  for (const target of targets) {
    const response = await compileGateway({ schema, target, config });
    results.set(target, response);
  }
}

```

```
    return results;
}
```

F-8.3 网关测试

```
// src/core/compiler/__tests__/compile-gateway.test.ts
```

```
import { describe, it, expect } from 'vitest';
import { compileGateway, compileMultiTarget } from '../gateway';
import type { CompileTarget } from '../targets';

const minimalSchema = {
  data: {
    formData: JSON.stringify({
      fields: [
        { __vModel__: 'name', __config__: { label: '姓名', tag: 'JnpfInput', jnpfKey:
'JnpfInput', required: true } },
        { __vModel__: 'age', __config__: { label: '年龄', tag: 'JnpfInputNumber',
jnpfKey: 'JnpfInputNumber' } },
      ],
      funcs: {},
      virtualFieldList: [],
    }),
  },
};
```

```
describe('统一编译网关', () => {
  it('Vue3 web 编译成功', async () => {
    const result = await compileGateway({
      schema: minimalSchema,
      target: 'vue3-web',
      config: { entity: 'student', entityLabel: '学生' },
    });
    expect(result.success).toBe(true);
    expect(result.project!.size).toBeGreaterThan(0);
  });

  it('UniApp 微信小程序编译成功', async () => {
    const result = await compileGateway({
      schema: minimalSchema,
      target: 'uniapp-weixin',
      config: { entity: 'student', entityLabel: '学生' },
    });
    expect(result.success).toBe(true);
    expect(result.project!.has('pages/student/list.vue')).toBe(true);
  });

  it('UniApp X App 编译成功', async () => {
    const result = await compileGateway({
      schema: minimalSchema,
      target: 'uniapp-x-app',
      config: { entity: 'student', entityLabel: '学生' },
    });
    expect(result.success).toBe(true);
    expect(result.project!.has('pages/student/list.uvue')).toBe(true);
  });

  it('错误 Schema 返回验证失败', async () => {
    const result = await compileGateway({
      schema: { data: { formData: '{}' } },
    });
```

```

    target: 'vue3-web',
    config: { entity: 'test' },
  });
  expect(result.success).toBe(false);
  expect(result.error).toContain('验证失败');
});

it('批量编译多个目标', async () => {
  const targets: CompileTarget[] = ['vue3-web', 'uniapp-weixin', 'uniapp-x-app'];
  const results = await compileMultiTarget(minimalSchema, targets, {
    entity: 'student',
    entityLabel: '学生',
  });

  expect(results.size).toBe(3);
  for (const [target, result] of results) {
    expect(result.success).toBe(true);
  }
});
});

```

Week 2: 下载源码 + ZIP 打包

F-8.4 ZIP 打包器

```

/**
 * ZIP 打包器
 *
 * 将 GeneratedProject (Map<filePath, content>) 打包为 ZIP 文件
 * 用户可下载 ZIP，在本地 IDE 中打开并运行
 *
 * 依赖: JSZip (轻量级 ZIP 库，浏览器端可用)
 */

import JSZip from 'jszip';
import type { GeneratedProject } from './vue3/types';

export interface ZipOptions {
  /** ZIP 文件名 */
  fileName?: string;
  /** 是否包含 README */
  includeReadme?: boolean;
  /** 是否包含 .gitignore */
  includeGitignore?: boolean;
  /** 是否包含安装说明 */
  includeInstallGuide?: boolean;
  /** 编译目标 (用于 README 内容) */
  target?: string;
  /** 实体名称 */
  entity?: string;
  /** 实体中文名 */
  entityLabel?: string;
}

/**
 * 将 GeneratedProject 打包为 ZIP
 */
export async function packToZip(

```

```

    project: GeneratedProject,
    options: ZipOptions = {}
  ): Promise<Blob> {
    const zip = new JSZip();
    const fileName = options.fileName ?? 'jnpf-generated-project';

    // 添加所有生成的文件
    for (const [filePath, content] of project) {
      zip.file(filePath, content);
    }

    // 添加 README
    if (options.includeReadme !== false) {
      zip.file('README.md', generateReadme(options));
    }

    // 添加 .gitignore
    if (options.includeGitignore !== false) {
      zip.file('.gitignore', generateGitignore());
    }

    // 添加安装指南
    if (options.includeInstallGuide !== false) {
      zip.file('INSTALL.md', generateInstallGuide(options));
    }

    // 生成 ZIP
    return zip.generateAsync({ type: 'blob' });
  }

/**
 * 下载 ZIP 文件（浏览器端）
 */
export function downloadZip(blob: Blob, fileName: string): void {
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = fileName;
  document.body.appendChild(a);
  a.click();
  document.body.removeChild(a);
  URL.revokeObjectURL(url);
}

function generateReadme(options: ZipOptions): string {
  return `# ${options.entityLabel ?? 'JNPF'} - 自动生成项目

> 由 JNPF 低代码平台代码生成器自动生成
> 编译目标: ${options.target ?? 'vue3-web'}
> 生成时间: ${new Date().toISOString()}

## 快速开始

\\\`bash
# 安装依赖
pnpm install

# 启动开发服务器
pnpm dev

```

构建生产版本

```
pnpm build
```

```
\\`\\`\\`
```

项目结构

此项目由 JNPF 代码生成器生成，可独立运行，不依赖 JNPF 平台。

- \@jnpf-generated\` 标记的文件由生成器管理
- \@jnpf-gen:insert-point\` 区域为安全的手动修改区域
- 重新生成时，insert-point 内的内容会被保留

二次开发

1. 在 \@jnpf-gen:insert-point\` 区域添加自定义代码
2. 新增的文件不会被重新生成覆盖
3. 如需回写到 JNPF 平台，使用平台的"导入源码"功能

技术栈

- Vue 3 + Composition API
 - TypeScript
 - Vite
 - Ant Design Vue / wot-design-uni
- ```
`;
}
```

```
function generateGitignore(): string {
 return `node_modules/
dist/
.DS_Store
*.local
.env.local
.env.*.local
*.log
`;
}
```

```
function generateInstallGuide(options: ZipOptions): string {
 const isUniApp = options.target?.startsWith('uniapp');

 if (isUniApp) {
 return `# UniApp 项目安装指南`
 }
}
```

## ## 环境要求

- Node.js >= 18
- HBuilder X (最新版)

## ## 安装步骤

1. 用 HBuilder X 打开此项目目录
2. 安装依赖: \@npm install\`
3. 运行到小程序: 菜单 → 运行 → 运行到小程序模拟器
4. 运行到 App: 菜单 → 运行 → 运行到 App

## ## 注意事项

- 小程序端不支持 eval/Function，所有逻辑已预编译
  - 如需修改业务逻辑，请在 \@jnpf-gen:insert-point\` 区域操作
- ```
`;  
}
```

```

    return `# web 项目安装指南

## 环境要求
- Node.js >= 18
- pnpm >= 8

## 安装步骤

1. 安装依赖: \`pnpm install\`
2. 启动开发服务器: \`pnpm dev\`
3. 浏览器打开 http://localhost:3100

## 部署

\\\`bash
# 构建
pnpm build

# 预览构建结果
pnpm preview
\\\`

构建产物在 \`${dist}/\` 目录, 可部署到任何静态服务器。
`;
}

```

F-8.5 下载源码 API (前端调用)

```

/**
 * 下载源码 API
 *
 * 前端页面调用此 API 触发编译 + 打包 + 下载
 */

import { compileGateway } from './gateway';
import { packToZip, downloadZip } from './zip-packer';
import type { CompileTarget, CompilerConfig } from './types';

export interface DownloadOptions {
  /** JSON Schema */
  schema: unknown;
  /** 编译目标 */
  target: CompileTarget;
  /** 编译配置 */
  config: CompilerConfig;
}

/**
 * 编译 + 打包 + 下载
 */
export async function downloadSourceCode(options: DownloadOptions): Promise<void> {
  // Step 1: 编译
  const result = await compileGateway({
    schema: options.schema,
    target: options.target,
    config: options.config,
  });

  if (!result.success || !result.project) {

```



```

    throw new Error(result.error ?? '编译失败');
  }

  // Step 2: 显示警告（如有）
  if (result.warnings && result.warnings.length > 0) {
    console.warn('[download] 编译警告:', result.warnings);
  }

  if (result.complexExpressions && result.complexExpressions.length > 0) {
    console.warn('[download] 以下表达式需人工迁移:', result.complexExpressions);
  }

  // Step 3: 打包 ZIP
  const zipBlob = await packToZip(result.project, {
    fileName: `jnpf-${options.config.entity}-${options.target}`,
    target: options.target,
    entity: options.config.entity,
    entityLabel: options.config.entityLabel,
    includeReadme: true,
    includeGitignore: true,
    includeInstallGuide: true,
  });

  // Step 4: 下载
  downloadZip(zipBlob, `jnpf-${options.config.entity}-${options.target}.zip`);
}

```

Week 3: 回写平台 + 平台 UI 集成

F-8.6 官方回写通道 (ir-to-schema, v5.0 必交付)

v5.0 裁定：废止「任意手改 .vue → IR」反解析器 (source-to-ir.ts / @vue/compiler-sfc 反解) 。官方回写 **唯一通道**：ir-to-schema.ts (IR → VisualDev JSON Schema) 。可选后续：@jnpf-block 受保护区块 + 字段级 diff UI，**不承诺** AST 全量反解。

```

/**
 * 官方回写通道 (v5.0)
 *
 * 路径：用户在 IDE 修改后 → 导入受控 IR 补丁 或 平台内 VisualDev 编辑
 *       → ir-to-schema.ts → VisualDev JSON Schema → 平台存储
 *
 * 废止：parseVueFileToIR / compiler-sfc 反解任意 .vue (工程无底洞，专家 E6)
 *
 * 逃生舱：Sprint 0-B 地桩 8 round-trip; 与 CompileGateway「导入 IR JSON」并列
 */

export { formPageIRToSchema } from '../..ir/ir-to-schema';
export type { WritebackResult } from '../..ir/ir-to-schema';

/** 平台 UI: 导入 IR JSON 或 Schema diff, 非上传任意 .vue 解析 */
export interface OfficialWritebackRequest {
  irPatch?: Partial<FormPageIR>;
  schemaOverride?: Record<string, unknown>;
  source: 'ir-json' | 'visual-dev';
}

```

F-8.7 平台 UI 集成设计

在 JNPF 表单设计器中新增"下载源码"按钮:

位置：表单设计器右上角操作栏

按钮：「[下载源码](#)」

弹窗：

选择编译目标	
○ Vue3 web 应用	[基础版]
○ 数字大屏	[基础版]
● 3D 数字孪生大屏	[VIP]
○ 微信小程序	[基础版]
○ 支付宝小程序	[基础版]
○ 抖音小程序	[基础版]
○ H5 移动端	[基础版]
○ 原生 App（标准 uni-app）	[基础版]
○ 原生 App（uni-app X）	[暂缓·灰显]
 □ 包含 3D 数字孪生（仅大屏有效）	
 实体名称：student 中文名称：学生管理 API 路径：/api/student	
[取消]	[下载源码]

"导入 IR / Schema"功能（官方回写，v5.0）：

位置：表单设计器 → 更多操作 → 导入 IR 或 Schema

弹窗:

官方回写（`ir-to-schema` 通道）

上传 `IR JSON` 或 `Schema diff` 文件
或 [在 `VisualDev` 中继续编辑]

⚠ 不支持上传任意 `.vue` 自动反解析

[取消] [应用变更]

F-8.8 下载源码弹窗组件 (前端 Vue3)

```
<!-- src/views/designer/DownloadSourceModal.vue -->
```

<!-- 在 JNPF 表单设计器中集成 -->

```
<template>
  <a-modal
    v-model:open="visible"
    title="下载源码"
    :width="600"
    :footer="null"
  >
    <div class="download-modal">
      <!-- 选择编译目标 -->
```

```

<div class="target-grid">
  <div
    v-for="target in availableTargets"
    :key="target.id"
    class="target-card"
    :class="{ selected: selectedTarget === target.id, vip: target.vip }"
    @click="selectedTarget = target.id"
  >
    <div class="target-icon">{{ target.icon }}</div>
    <div class="target-name">{{ target.name }}</div>
    <div class="target-desc">{{ target.description }}</div>
    <div v-if="target.vip" class="vip-badge">VIP</div>
  </div>
</div>

<!-- 配置 -->
<a-form layout="vertical" style="margin-top: 24px">
  <a-form-item label="实体名称（英文）">
    <a-input v-model:value="config.entity" placeholder="如： student" />
  </a-form-item>
  <a-form-item label="中文名称">
    <a-input v-model:value="config.entityLabel" placeholder="如： 学生管理" />
  </a-form-item>
  <a-form-item label="API 基础路径">
    <a-input v-model:value="config.apiBasePath" placeholder="如： /api/student" />
  </a-form-item>
</a-form>

<!-- 警告信息 -->
<a-alert
  v-if="warnings.length > 0"
  type="warning"
  show-icon
  style="margin-top: 16px"
>
  <template #message>
    <div v-for="warn in warnings" :key="warn">{{ warn }}</div>
  </template>
</a-alert>

<!-- 操作按钮 -->
<div class="modal-footer">
  <a-button @click="visible = false">取消</a-button>
  <a-button type="primary" :loading="loading" @click="handleDownload">
    下载源码
  </a-button>
</div>
</div>
</a-modal>
</template>

<script setup lang="ts">
import { ref, computed, watch } from 'vue';
import { message } from 'ant-design-vue';
import { COMPILER_TARGETS, type CompileTarget } from '@core/compiler/targets';
import { downloadSourceCode } from '@core/compiler/download';
import { useDesignerStore } from '@stores/designer';

const visible = ref(false);
const loading = ref(false);

```

```

const selectedTarget = ref<CompileTarget>('vue3-web');
const warnings = ref<string[]>([]);

const config = ref({
  entity: '',
  entityLabel: '',
  apiBasePath: '',
});

// 可用目标 (VIP 检查)
const availableTargets = computed(() => {
  const isvip = useDesignerStore().isvipLicense;
  return Object.values(COMPILE_TARGETS).filter(t => !t.vip || isvip);
});

// 自动填充实体名称
watch(visible, (val) => {
  if (val) {
    const store = useDesignerStore();
    config.value.entity = store.currentForm?.enCode ?? '';
    config.value.entityLabel = store.currentForm?.fullName ?? '';
    config.value.apiBasePath = `/api/${config.value.entity}`;
  }
});

async function handleDownload() {
  if (!config.value.entity) {
    message.warning('请输入实体名称');
    return;
  }

  loading.value = true;
  try {
    const store = useDesignerStore();
    await downloadSourceCode({
      schema: store.currentSchema,
      target: selectedTarget.value,
      config: {
        entity: config.value.entity,
        entityLabel: config.value.entityLabel,
        apiBasePath: config.value.apiBasePath,
      },
    });
    message.success('源码下载成功');
    visible.value = false;
  } catch (e) {
    message.error(`下载失败: ${e as Error}.message`);
  } finally {
    loading.value = false;
  }
}

defineExpose({ open: () => { visible.value = true; } });
</script>

<style scoped>
.target-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 12px;

```

```
}
.target-card {
  position: relative;
  padding: 16px;
  border: 2px solid #f0f0f0;
  border-radius: 8px;
  cursor: pointer;
  text-align: center;
  transition: all 0.2s;
}
.target-card:hover {
  border-color: #0083ff;
}
.target-card.selected {
  border-color: #0083ff;
  background: #f0f7ff;
}
.target-card.vip {
  border-color: #faad14;
}
.target-card.vip.selected {
  background: #fffbe6;
}
.vip-badge {
  position: absolute;
  top: 4px;
  right: 4px;
  background: #faad14;
  color: #fff;
  font-size: 10px;
  padding: 2px 6px;
  border-radius: 4px;
}
.target-icon {
  font-size: 24px;
  margin-bottom: 8px;
}
.target-name {
  font-weight: 600;
  font-size: 13px;
}
.target-desc {
  font-size: 11px;
  color: #999;
  margin-top: 4px;
}
.modal-footer {
  display: flex;
  justify-content: flex-end;
  gap: 12px;
  margin-top: 24px;
}
</style>
```

阶段四测试

```
// src/core/compiler/__tests__/platform-integration.test.ts

import { describe, it, expect } from 'vitest';
import { compileGateway, compileMultiTarget } from '../gateway';
import { packToZip } from '../zip-packer';
import { formPageIRToSchema } from '../ir/ir-to-schema';
import { COMPILE_TARGETS } from '../targets';

const minimalSchema = {
  data: {
    formData: JSON.stringify({
      fields: [
        { __vModel__: 'name', __config__: { label: '姓名', tag: 'JnpfInput', jnpfkey:
'JnpfInput' } } },
      ],
      funcs: {},
      virtualFieldList: [],
    }),
  },
};

describe('平台整合', () => {

  describe('编译目标完整性', () => {
    it('所有编译目标都有元数据', () => {
      const targets = Object.keys(COMPILE_TARGETS);
      expect(targets.length).toBe(8);
      for (const target of targets) {
        const meta = COMPILE_TARGETS[target as keyof typeof COMPILE_TARGETS];
        expect(meta.name).toBeTruthy();
        expect(meta.description).toBeTruthy();
      }
    });
  });

  describe('ZIP 打包', () => {
    it('打包后包含 README', async () => {
      const result = await compileGateway({
        schema: minimalSchema,
        target: 'vue3-web',
        config: { entity: 'test', entityLabel: '测试' },
      });

      const blob = await packToZip(result.project!, {
        target: 'vue3-web',
        entity: 'test',
      });

      expect(blob).toBeInstanceOf(Blob);
      expect(blob.size).toBeGreaterThan(0);
    });
  });

  describe('源码回写', () => {
    it('解析 .vue 文件提取字段', () => {
      const vueContent = `
<template>
```

```

<a-form>
  <a-form-item label="姓名" name="name">
    <a-input v-model:value="formData.name" />
  </a-form-item>
  <a-form-item label="年龄" name="age">
    <a-input-number v-model:value="formData.age" />
  </a-form-item>
</a-form>
</template>
<script setup lang="ts">
// @jnpf-gen:insert-point=custom-logic
// @jnpf-gen:end-insert-point=custom-logic
</script>`;

    const result = parseVueFileToIR(vueContent);
    expect(result.canWriteback).toBe(true);
    expect(result.ir!.fields!.length).toBe(2);
    expect(result.ir!.fields![0].model).toBe('name');
    expect(result.ir!.fields![0].component!.jnpfKey).toBe('JnpfInput');
    expect(result.ir!.fields![1].component!.jnpfKey).toBe('JnpfInputNumber');
  });

  it('比较原始 IR 和修改后的 IR', () => {
    const original = {
      type: 'form' as const,
      id: 'test',
      name: 'test',
      fields: [
        { id: '1', model: 'name', label: '姓名', component: { jnpfKey: 'JnpfInput',
pc: 'a-input', app: 'uni-easyinput' }, config: {} as any, validation: [], events: {} },
        { id: '2', model: 'age', label: '年龄', component: { jnpfKey:
'JnpfInputNumber', pc: 'a-input-number', app: 'uni-number-box' }, config: {} as any,
validation: [], events: {} },
      ],
      config: {} as any,
      databaseFields: [],
      expressions: [],
    };

    const modified = {
      fields: [
        { model: 'name', label: '用户姓名', component: { jnpfKey: 'JnpfInput', pc: '',
app: '' } }, // 修改了 label
        // age 被删除
        { model: 'email', label: '邮箱', component: { jnpfKey: 'JnpfInput', pc: '',
app: '' } }, // 新增
      ] as any[],
    };

    const diff = diffIR(original, modified);
    expect(diff.addedFields.length).toBe(1); // email
    expect(diff.removedFields.length).toBe(1); // age
    expect(diff.modifiedFields.length).toBe(1); // name label changed
  });
});
});
});

```

阶段四交付物

- `src/core/compiler/targets.ts` - 编译目标枚举 + 元数据
- `src/core/compiler/gateway.ts` - 统一编译网关
- `src/core/compiler/zip-packer.ts` - ZIP 打包器
- `src/core/compiler/download.ts` - 下载源码 API
- `src/core/ir/ir-to-schema.ts` - 官方回写通道 (IR → Schema)
- `src/views/designer/DownloadSourceModal.vue` - 下载源码弹窗组件
- `src/views/designer/ImportIrModal.vue` - 导入 IR/Schema 回写弹窗
- `src/core/compiler/__tests__/compile-gateway.test.ts` - 网关测试
- `src/core/compiler/__tests__/platform-integration.test.ts` - 整合测试
- 标签: `v5.2-platform-integration-m1`

阶段四里程碑验收

- 统一编译网关支持全部 8 个编译目标
- 批量编译 (multi-target) 成功
- ZIP 打包器正确生成可下载的 ZIP 文件
- 下载的 ZIP 包含 README + .gitignore + INSTALL.md
- 官方回写: `ir-to-schema round-trip` 通过 (非 .vue 反解析)
- 导入 IR JSON / Schema diff UI 完成
- 所有测试通过 (网关 + 整合 + 回写)
- 零 eval/Function

四个阶段的完整交付物总览

阶段零 (已完成):

- ✓ F-0 安全止血 (7 文件, eval/Function 归零)
- ✓ F-1 IR 类型系统 + Schema 清洗器 (6 文件)
- ✓ F-2 安全表达式引擎 (11 文件, 35 测试)
- ✓ F-3 组件注册表 (5 文件, 35 组件)
- ✓ F-4 Vue3 编译器 (8 文件, 8 测试)
- ✓ 知识图谱文档 4 份 + ADR-016

阶段一 (4 周):

- F-5 端到端验证 + 演示项目 + ESLint 规则
- F-6a 大屏 IR + 大屏编译器 (基础模块)

阶段二 (4 周):

- F-6b 3D 数字孪生 VIP (事件绑定 DSL 并入表达式引擎)
- 后端清零 Sprint 4-5

阶段三 (4 周):

- F-7 UniApp **单轨** + FlowIR v1
- F-7 Alova + Pinia + pages.json 合并器
- 后端清零收尾 (业务层 App.GetService, 非 Furion 框架层)

阶段四 (3 周):

- F-8 统一编译网关 (8 个目标)
- F-8 ZIP 打包 + 下载源码
- F-8 **ir-to-schema 官方回写**
- F-8 平台 UI 集成 (下载 + 导入 IR 弹窗)

总计:

- 约 50+ 个源文件
- 约 100+ 个测试用例
- 8 个编译目标
- 后端完全清零

阶段一到阶段四（16 周）是"手工低代码平台的顶峰一跃"。完成后，JNPF 将拥有：

- **Web 端代码生成** (Vue3 + Ant Design Vue)
- **大屏代码生成** (ECharts + 装饰 + 3D 数字孪生)
- **小程序代码生成** (微信/支付宝/抖音)
- **App 代码生成** (uni-app X 原生)
- **下载源码能力** (用户可脱离平台运行)
- **回写平台能力** (双向同步)
- **后端完全清零** (零技术债务)
-